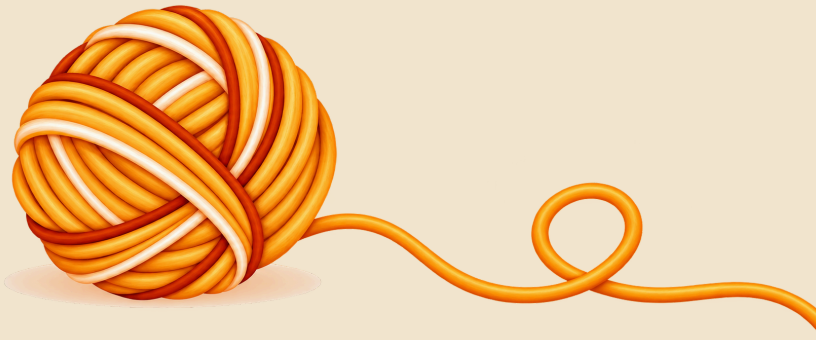
TRAMA

Distintas miradas, una misma persona

Informe de pruebas de carga, capacidad y seguridad

Qué aguanta TRAMA hoy, qué la satura, qué tan escalable es, cuánto entra en 50 GB y cómo arreglar lo que encontramos



6 de noviembre de 2026

Pruebas realizadas el 5 y 6 de noviembre de 2026 sobre la rama **develop**

Todos los datos usados son inventados. Documento interno del equipo de TRAMA.

Contenido

1 **Resumen y prioridades**
Qué servidor conviene, qué satura a TRAMA y qué arreglar primero

2 **Cómo testeamos**
Entorno, simulación de servidores, cada prueba paso a paso y cómo se midió

3 **Rendimiento**
El techo de cada servidor y la cadena que lo satura

4 **Casos límite**
Muchos aportes, ediciones simultáneas, archivos, logins y disco lleno

5 **Capacidad en 50 GB**
Cuánto ocupa cada cosa y cuánto entra

6 **Seguridad**
Código, configuración y dependencias

7 **Escalabilidad**
Qué escala, qué no, por qué y cómo podría escalar

8 **Cómo arreglar las vulnerabilidades**
Tres etapas, con código de ejemplo y cómo verificar cada arreglo

A **Anexos**
Glosario, consultas SQL, módulos, excepciones y versiones

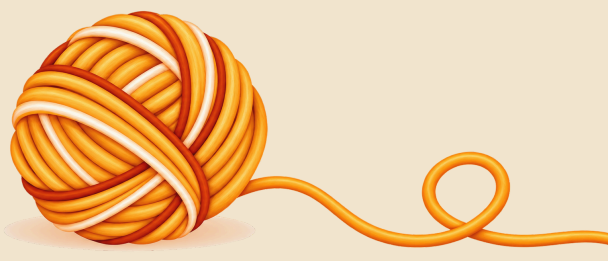
Cómo leer este informe. El capítulo 1 alcanza para decidir. Los capítulos 2 a 6 tienen la evidencia de cada afirmación; el 7 analiza la escalabilidad y el 8 explica cómo arreglar cada hallazgo. Los anexos incluyen un glosario de los términos técnicos.

TRAMA

CAPÍTULO 1

Resumen y prioridades

Qué servidor conviene, qué satura a TRAMA y qué arreglar primero



162

pedidos por segundo

Techo de 1 CPU · 2 GB con uso normal. El de 2 CPU · 4 GB llega a 282.

75 %

de la CPU en BCrypt

Con el servidor saturado. El login retiene una conexión a la base mientras calcula.

1.000

aportes por niño

Desde ahí abrir un perfil tarda 3,7 s en 1 CPU · 2 GB. Las listas no tienen paginación.

~1.150

niño-años en 50 GB

Con los backups en otro lado. Con 7 backups en el mismo disco, ~150.

806

errores de memoria

Subidas y descargas que fallan después de un pico de uso, sin rastro en el log.

13

vulnerabilidades críticas

En la imagen del backend, por Spring Boot 3.1 sin soporte.

1.1 Para qué sirve este informe

Para decidir **qué servidor pagar**, saber **cuánto aguanta TRAMA hoy**, **qué la satura exactamente** y **qué arreglar primero** antes de salir a producción.

1.2 Respuestas cortas

¿Qué servidor conviene? Un **VPS de 1 CPU y 2 GB** alcanza para empezar, siempre que se arreglen los puntos de la tabla de prioridades. El de **1 GB no sirve**: con el compose actual MySQL ni siquiera arranca, y ajustado se cae con 50 personas. El de **2 CPU y 4 GB** da cerca del doble de techo, pero no resuelve los problemas de diseño.

Servidor	¿Arranca?	Techo medido (pedidos/s)	p95 con 100 personas	Veredicto
1 CPU · 1 GB (compose actual)	No MySQL muere por memoria al iniciar	—	—	No sirve
1 CPU · 1 GB (ajustado)	Sí	se cae con 50 personas (sin memoria)	—	No sirve
1 CPU · 2 GB	Sí	**118** (con re-logins) · **162** (login una vez)	279 ms · 186 ms	Alcanza para empezar
2 CPU · 4 GB	Sí	**196** (con re-logins) · **282** (login una vez)	24 ms · 14 ms	Holgado; mismos límites de diseño

¿Qué satura a TRAMA? Una cadena de tres eslabones (detalle en capítulo 3 (Rendimiento)):

1. **BCrypt** (la verificación de contraseñas) se come entre el **57 % y el 78 % de la CPU** cuando hay logins.
2. Por **Open Session In View** (activado por defecto en Spring y no desactivado en TRAMA), el login **retiene una de las 10 conexiones a la base mientras calcula BCrypt**.
3. Con la CPU ocupada, las 10 conexiones se agotan y **todos los pedidos hacen cola** hasta 30 s. En un minuto de saturación los hilos sumaron **10.671 segundos esperando una conexión**.

En segundo plano: el **filtro JWT** (11 % a 24 % de la CPU de cada pedido: valida el token tres veces y busca al usuario en la base) y unas **27 consultas SQL por pedido** (N+1 en adjuntos y permisos repetidos). **MySQL no es el cuello de botella**: ninguna consulta pasó de 0,2 ms en promedio.

¿Cuál es el límite real? No es la cantidad de usuarios ni el disco: es cuántos aportes tiene cada niño. La lista de aportes no tiene paginación. Con **1.000 aportes por niño** (menos de un año de uso activo), abrir un perfil tarda **3,7 s** en 1 CPU · 2 GB con 20 personas mirando. Con 200.000 aportes en la base (unos 1.000 por niño), la misma carga que antes respondía en 279 ms pasa a **15,3 s**. Ver capítulo 4 (Casos límite).

¿Cuánto entra en 50 GB? Con los supuestos de uso de capítulo 5 (Capacidad en 50 GB), quedan unos **30 GB útiles** para datos. Cada niño ocupa unos **26 MB por año**, el 93 % por archivos adjuntos. Eso da para **~1.150 niño-años** (por ejemplo, 575 niños durante 2 años) **si los backups van a otro lado**. Con 7 backups diarios en el mismo disco baja a **~150 niño-años**, porque cada backup copia **todos** los adjuntos y el script actual **no borra backups viejos**.

1.3 Hallazgos y prioridades

Ordenados por impacto. Cada uno tiene la evidencia en el documento que se indica.

#	Hallazgo	Impacto	Dónde
1	La lista de aportes y la bitácora traen **todo** sin paginar, con una consulta extra por aporte (N+1 de adjuntos) y descifrando cada texto	Con ~1.000 aportes por niño la app se vuelve lenta; con 20.000 deja de responder	capítulo 4 (Casos límite)
2	**Subidas y descargas fallan con `500` después de un pico de uso**: `ArchivoService` lee y escribe cada archivo entero y Java agota la *memoria directa*	Entre 26 % y 75 % de subidas fallidas hasta reiniciar	capítulo 4 (Casos límite)
3	**Disco lleno = TRAMA caída**: MySQL se queda esperando espacio, se agotan las conexiones y no se recupera sola	Caída total, intervención manual	capítulo 4 (Casos límite)
4	`backup.sh` **no borra backups viejos** y cada uno copia todos los adjuntos	El disco se llena por los backups	capítulo 5 (Capacidad en 50 GB)
5	Open Session In View activado: el login retiene la conexión durante BCrypt	Saturación brusca: todos los pedidos esperan	capítulo 3 (Rendimiento)
6	`docker-compose.yml` publica MySQL (`3307`) y el backend (`4002`) en todas las interfaces	En un VPS quedarían abiertos a internet (Docker se saltea `ufw`)	capítulo 6 (Seguridad)
7	La demo pública corre con el perfil `dev` : `DataSeeder` crea cuentas con contraseña fija `Trama1234!`	Cualquiera entra a la demo con esas cuentas	capítulo 6 (Seguridad)
8	Dependencias sin soporte: **13 vulnerabilidades críticas** en la imagen del backend (Tomcat 10.1.20, Spring 6.0/6.1)	Varias aplican (BCrypt de 72 bytes, DoS en subidas)	capítulo 6 (Seguridad)
9	Dos ediciones simultáneas del mismo aporte producen *deadlock* en MySQL y la persona recibe `500`	61 % de fallas con 10 ediciones a la vez (sin pérdida de datos)	capítulo 4 (Casos límite)
10	`GlobalExceptionHandler` devuelve `500` sin escribir el error en el log	Los errores de producción quedan invisibles	capítulo 6 (Seguridad)
11	Sin límite de pedidos por IP en login, registro y recuperación	Bloqueo de cuentas ajenas, abuso de mails, saturación por BCrypt	capítulo 6 (Seguridad)

#	Hallazgo	Impacto	Dónde
12	El **título** de los aportes se guarda sin cifrar (`aporte.titulo`, `aporte_version.titulo`)	Contradice la regla de cifrar todo texto libre sobre el niño	capítulo 6 (Seguridad)

¿Es escalable? En cantidad de familias y de datos, sí. Agrandando el servidor, a medias (+66 % a +74 % con el doble de CPU). En **aportes por niño**, no, y hoy **no admite más de una copia del backend** (archivos en el disco local y tareas programadas que se repetirían). Ver capítulo 7 (Escalabilidad).

Qué haría primero (sin tocar reglas de negocio)

Prioridad	Cambio	Resuelve
Antes de producción	Paginar `GET /aportes` y `GET /bitacora`, y traer los adjuntos con una sola consulta	1
Antes de producción	Leer y escribir archivos por partes (*streaming*), o como paliativo `-Djdk.nio.maxCachedBufferSize=262144`	2
Antes de producción	Alertar cuando el disco pase el 80 %, rotación de backups y `binlog_expire_logs_seconds` menor, o binlogs apagados si no hay réplica	3, 4
Antes de producción	Publicar puertos solo en `127.0.0.1`, la demo sin `DataSeeder`	6, 7
Antes de producción	Actualizar Spring Boot a una versión con soporte (arrastra Tomcat, Spring Security y Jackson)	8
Pronto	`spring.jpa.open-in-view=false` y revisar qué servicios necesitan la sesión abierta	5
Pronto	Límite de pedidos por IP en `/api/auth/*`	11
Pronto	Registrar en el log los errores inesperados; `409` con mensaje claro en vez de `500` ante un *deadlock*	9, 10
Para decidir con el equipo	Cifrar el título del aporte (cambio de esquema: requiere migración y migrar los datos existentes)	12

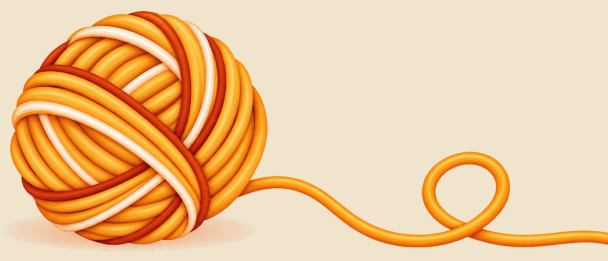
El detalle de cada arreglo, con código y cómo verificarlo, está en capítulo 8 (Cómo arreglar las vulnerabilidades).

TRAMA

CAPÍTULO 2

Cómo testeamos

Entorno, simulación de servidores, cada prueba paso a paso y cómo se midió



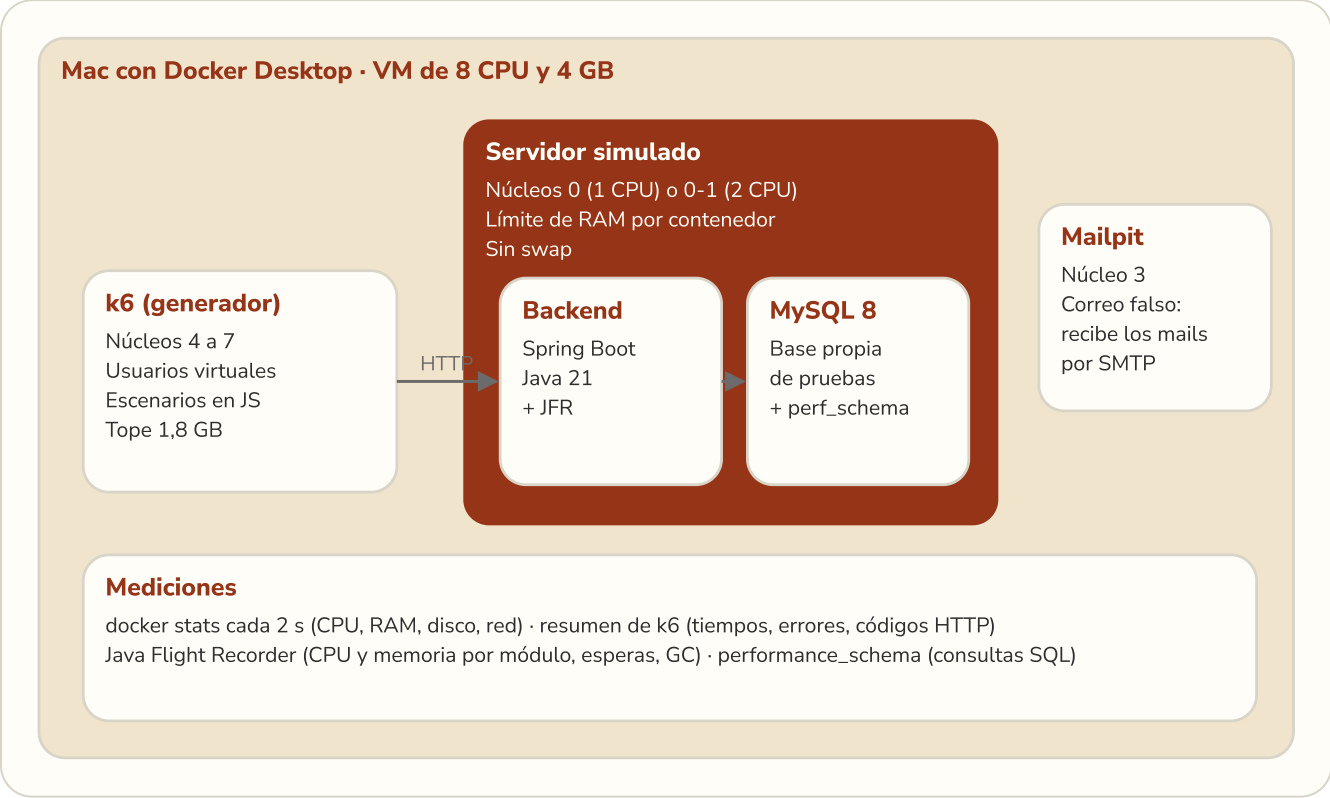
2.1 Objetivo

Responder cinco preguntas con mediciones, no con estimaciones:

1. ¿Qué servidor conviene pagar y cuánto aguanta cada uno?
2. ¿Qué es **exactamente** lo que satura a TRAMA?
3. ¿Qué pasa en los casos límite: niños con muchos aportes, picos de logins, archivos, disco lleno?
4. ¿Cuánto entra en un disco de 50 GB?
5. ¿Qué vulnerabilidades tiene hoy?

2.2 El entorno

Todo corrió en una Mac (Apple Silicon, 8 núcleos, 8 GB) con Docker Desktop, en una carpeta **fuera del repo** (`~/Documents/Trama-pruebas`), con su propio compose, su propia base y sus propias claves. El repo, la base de desarrollo y el `docker-compose.yml` no se tocaron.



Cómo se armó el entorno de pruebas

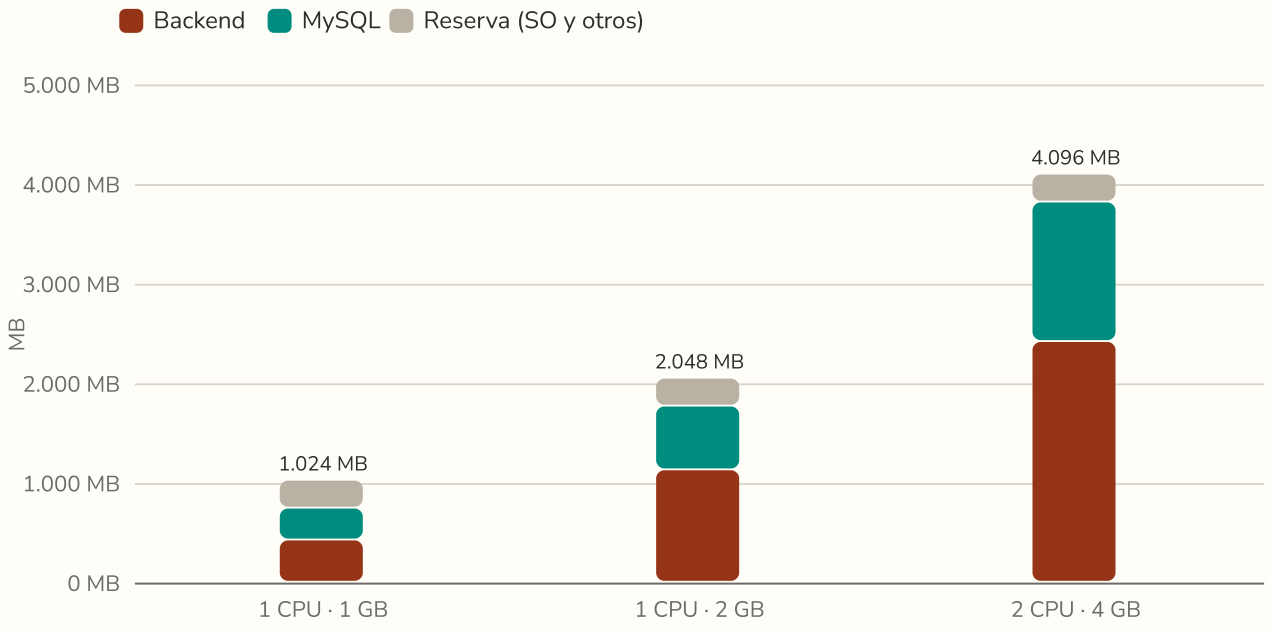
Pieza	Detalle
Backend	La imagen del repo (`trama-backend:local`, compilada desde `develop`): Spring Boot 3.1.11, Java 21. Perfil `dev` con las claves por variable de entorno; `prod` solo cambia de dónde salen las claves (OpenBao)
Base	MySQL 8.0 con los mismos parámetros del compose del repo
Generador de carga	k6 0.54.0 en Docker. Cada escenario es un script en JavaScript

Pieza	Detalle
Mails	Mailpit, un servidor de correo falso. Ningún mail salió a internet
Aislamiento	El "servidor" usa los núcleos 0 y 1; k6 usa del 4 al 7 y Mailpit el 3. Así el generador no le roba CPU al servidor que se mide

2.3 Cómo se simuló cada servidor

Un VPS barato tiene todo en la misma máquina: MySQL, el backend, OpenBao, nginx y el sistema operativo compartiendo CPU y memoria. Para reproducirlo:

- **CPU:** MySQL y el backend se fijaron a los **mismos** núcleos (`cpuset`): uno para los servidores de 1 CPU y dos para el de 2 CPU. Compiten entre sí como en un VPS real.
- **Memoria:** cada contenedor tiene un tope y **no tiene swap**. Si lo pasa, el sistema lo mata, como pasaría en el servidor.
- **Reserva:** se dejaron unos 280 MB para el sistema operativo, Docker, OpenBao y nginx.



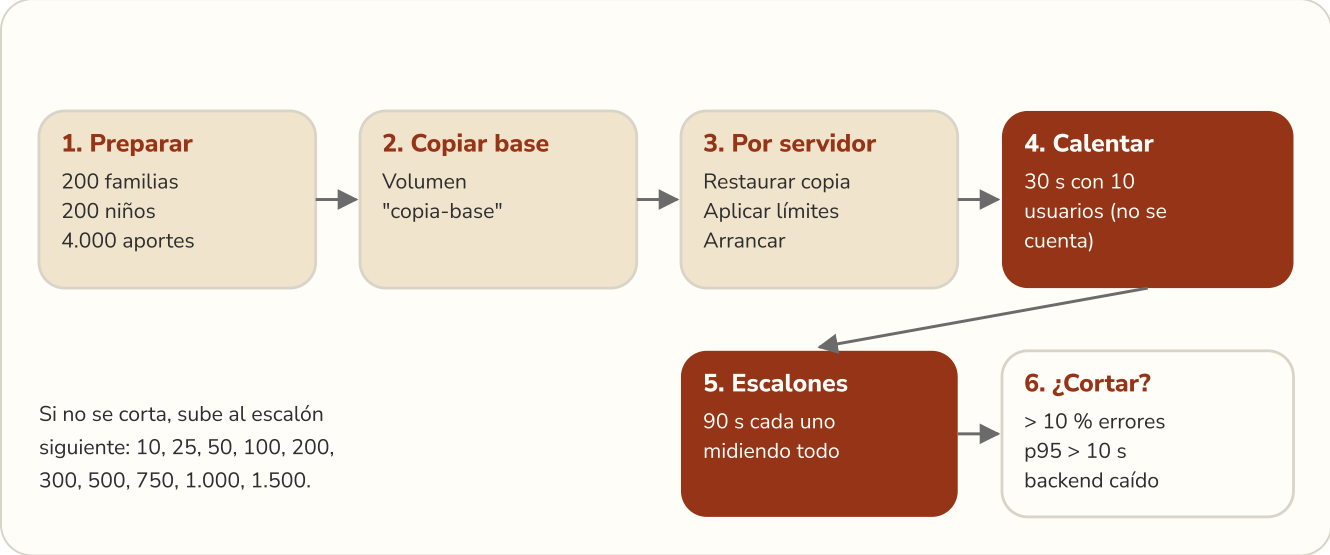
Reparto de memoria de cada servidor simulado

Servidor	Núcleos	RAM de MySQL	RAM del backend	Ajustes
1 CPU · 1 GB	1	320 MB	424 MB	Ninguno (como el compose del repo)
1 CPU · 1 GB ajustado	1	320 MB	424 MB	MySQL sin `performance_schema` y buffer de 64 MB; Java con heap al 70 %
1 CPU · 2 GB	1	640 MB	1.128 MB	Ninguno
2 CPU · 4 GB	2	1.400 MB	2.416 MB	Ninguno

2.4 Datos de prueba

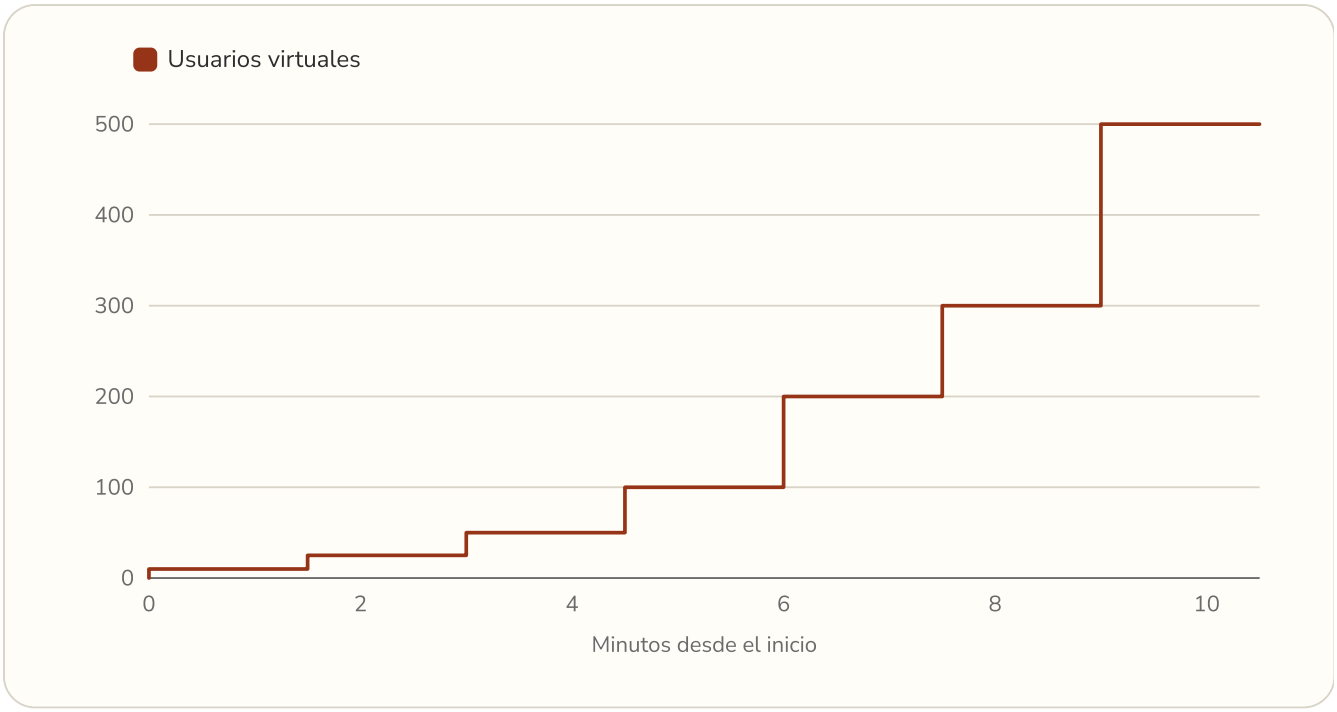
- **200 familias inventadas** (`familia1@prueba.trama.test` a `familia200@…`), cada una con un niño y 20 aportes: 4.000 aportes entre observaciones y estrategias. Todos los textos son ficticios.
- Las cuentas se crearon por la API (`POST /api/auth/registro`), igual que una persona. Solo se marcó el email como verificado directamente en la base, para no tener que leer 200 códigos.
- Terminada la carga, se guardó una **copia de la base**. Antes de cada servidor se restaura esa copia, así todos arrancan exactamente iguales.

2.5 El proceso de cada prueba



Proceso de cada prueba

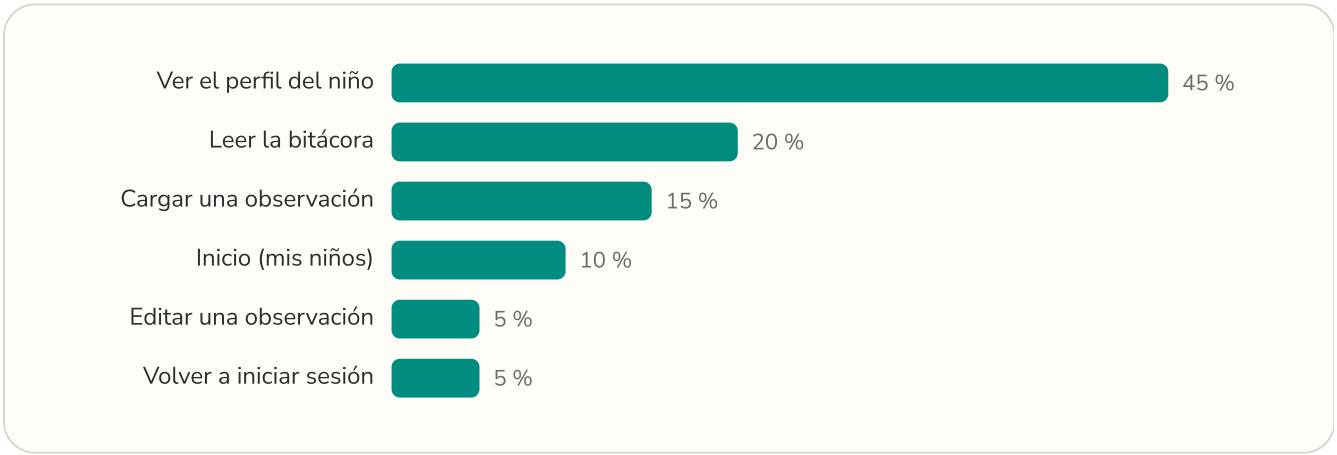
1. **Restaurar** la copia de la base y levantar los contenedores con los límites del servidor.
2. **Calentar** 30 segundos con 10 usuarios. Java compila el código más usado en los primeros minutos; sin calentamiento, el primer escalón saldría peor de lo que es.
3. **Subir de a escalones** de 90 segundos. En cada uno se guarda el resumen de k6 y se mide CPU, memoria, disco y red cada 2 segundos.
4. **Cortar** cuando hay más del 10 % de errores, el p95 pasa los 10 segundos o el backend deja de responder. Subir más no agrega información.



La escalera de usuarios a lo largo de la prueba

2.6 El escenario de uso mezclado

Cada usuario virtual es una familia que inicia sesión, toma el niño de su cuenta y repite estas acciones al azar, con pausas de 1 a 3 segundos:



Qué hace cada usuario virtual

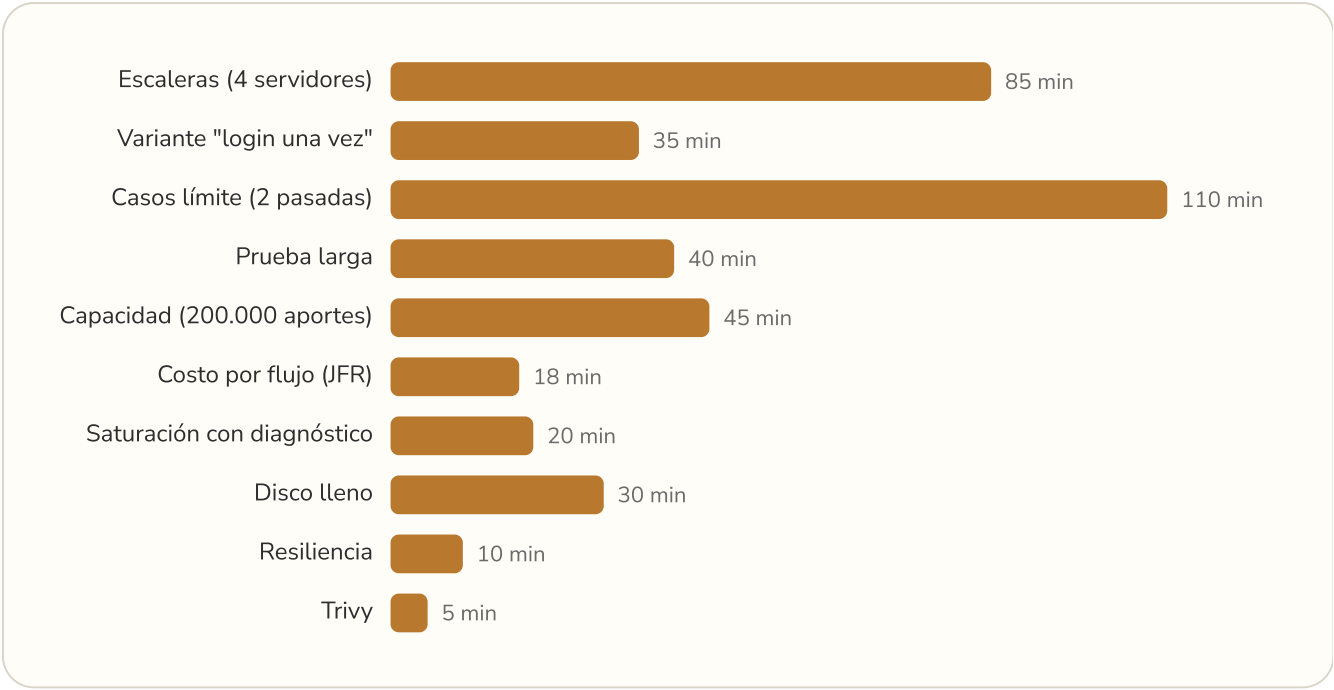
Acción	Peso	Pedidos
Ver el perfil del niño	45 %	`GET /api/proyectos/{id}` + `GET /aportes`
Leer la bitácora	20 %	`GET /bitacora`
Cargar una observación	15 %	`POST /aportes`
Inicio	10 %	`GET /api/proyectos` + `GET /notificaciones/no-leidas/count`
Editar una observación propia	5 %	`PUT /aportes/{id}`

Acción	Peso	Pedidos
Volver a iniciar sesión	5 %	`POST /api/auth/login`

Para separar el peso de BCrypt se repitió la escalera con la **variante "login una vez"**: cada usuario inicia sesión al empezar y su 5 % de re-login pasa a ver aportes.

2.7 Las pruebas, una por una

Prueba	Cómo se hizo	Qué se buscaba
Escalera hasta saturar	Uso mezclado de 10 a 1.500 usuarios, 4 servidores	Techo de cada servidor
Variante "login una vez"	La misma escalera sin re-logins, en 2 GB y 4 GB	Cuánto pesa BCrypt
Costo por flujo	Cada flujo solo, a ritmo fijo (20 pedidos/s; 8 para login y 5 para archivos), 60 s grabados con Java Flight Recorder	CPU y memoria de cada módulo
Saturación con diagnóstico	Uso mezclado en el codo (200 y 300 usuarios) y pasado el techo (400 y 600), con JFR y `performance_schema`	Qué satura exactamente
Prueba larga	20 minutos con 100 usuarios	Fugas de memoria
Resiliencia	Con 50 usuarios activos, MySQL se apaga 30 s y se vuelve a prender	Si TRAMA se recupera sola
Tormenta de logins	De 1 a 80 logins por segundo en 3 minutos	Fuerza bruta o entrada masiva
Ediciones simultáneas	10, 50 y 200 personas guardando **el mismo** aporte a la vez; después se cuentan las versiones en la base	Integridad de datos
Textos largos	50 personas cargando aportes de 10.000 caracteres	El máximo permitido
Archivos	5, 20 y 50 personas subiendo y bajando archivos de 5 MB	Memoria y disco
Niño con muchos aportes	Se cargaron 1.000, 5.000 y 20.000 aportes en un niño; 20 personas miran su perfil	Listas sin paginar
Capacidad	200.000 aportes, 100 archivos y un volcado de la base, midiendo el disco en cada paso	Cuánto ocupa cada cosa
Disco lleno	MySQL en un disco de 700 MB que se llena cargando aportes; después se intenta liberar espacio	Qué pasa y si se recupera
Enumeración de cuentas	20 logins y 20 recuperaciones con emails registrados y 20 con inexistentes, una por cuenta	Si el tiempo revela quién tiene cuenta
Dependencias	Trivy sobre las imágenes del backend y el frontend; `npm audit`	Vulnerabilidades conocidas



Tiempo dedicado a cada tipo de prueba

2.8 Cómo se midió

Herramienta	Qué mide	Cada cuánto
k6	Tiempo de cada pedido (mediana, p95, p99, máximo), pedidos por segundo, errores y **cantidad de respuestas por código HTTP**	Todos los pedidos
docker stats	CPU, memoria, disco leído y escrito, y red de cada contenedor	Cada 2 s
docker inspect	Si el contenedor se reinició o lo mató el sistema por falta de memoria (`OOMKilled`)	Al final de cada escalón
Java Flight Recorder	Qué código usa la CPU, cuánta memoria pide cada método, esperas de conexiones, bloqueos, pausas del recolector de basura y excepciones	Grabaciones de 60 s
performance_schema	Cada consulta SQL: cuántas veces se ejecutó, tiempo total, filas examinadas, uso de índices	Por prueba
SHOW GLOBAL STATUS	Caché de MySQL, conexiones, bloqueos, tablas temporales	Por prueba
du / df	Tamaño de tablas, binlogs y archivos; espacio libre	Por paso

Las grabaciones de JFR se procesaron con un script propio que agrupa cada muestra por **capa** (BCrypt, Hibernate, driver de MySQL, Jackson, Tomcat, Spring, JWT, cifrado) y por **módulo de TRAMA** (filtro JWT, aportes, permisos, login, archivos, etc.), mirando qué código estaba en la pila en ese momento.

2.9 Cómo cuidamos que los números sean confiables

Errores de nuestros propios scripts, encontrados y corregidos:

Problema detectado	Cómo se notó	Qué se hizo
El script editaba *estrategias* con datos de *observación* y TRAMA lo rechazaba (bien)	0,2 % a 0,7 % de errores incluso con 10 usuarios	Editar solo observaciones y repetir todas las escaleras
La carga de 20.000 aportes no se había hecho (error de configuración de k6)	La lista con 20.000 aportes respondía igual que con 1.000	Cargarlos en paralelo, contar en la base y repetir
Los errores de una prueba se mezclaban con los de la anterior en el log	Deadlocks que aparecían en la prueba de archivos	Leer el log desde el inicio exacto de cada prueba
El conteo de versiones daba 0 por la tilde de "Edición"	Resultado imposible	Cliente de MySQL en UTF-8
La prueba de resiliencia se cortaba cuando `curl` agotaba su espera	Faltaba el dato de recuperación	Tolerar el corte y repetir

Otras precauciones:

- **Misma base para todos:** cada servidor arrancó desde la misma copia.
- **Calentamiento descartado:** los primeros 30 s no se cuentan.
- **Generador separado:** k6 en otros núcleos y con su propio tope de memoria (1,8 GB), para que si algo se queda sin memoria sea el generador y no el "servidor".
- **Causas confirmadas, no supuestas:** los 500 de los archivos se reprodujeron dos veces y la causa se confirmó con JFR; la enumeración de cuentas se midió, no se dedujo del código.
- **Tope de disco:** la prueba de capacidad usó un disco virtual con un máximo de 35 GB y un control que frenaba la carga si Docker más los discos de prueba pasaban de 45 GB. El uso real máximo fue de 11 GB.

2.10 Cómo leer los números

- **p95**: el 95 % de los pedidos tardó menos que eso. Muestra la experiencia de las personas con peor suerte.
- **Techo**: el máximo de pedidos por segundo que atendió el servidor. Pasado ese punto, más gente se traduce en más espera.
- **Usuario virtual no es una persona.** Un usuario virtual hace una acción cada 1 a 3 segundos; una persona real tarda bastante más entre clics. Conviene pensar en **pedidos por segundo**:

> **Personas activas a la vez ≈ pedidos/s × segundos entre acciones × factor de CPU del VPS**
>
> Con 10 s entre acciones y un factor de 0,5, el techo de 1 CPU · 2 GB (160 pedidos/s) equivale a unas **800 personas usando la app en el mismo momento**, y el de 2 CPU · 4 GB (280 pedidos/s) a unas **1.400**. "Usando en el mismo momento" no es lo mismo que "registradas": suele ser una fracción chica.

2.11 Limitaciones

Limitación	Efecto
Los núcleos de Apple Silicon son más rápidos que un vCPU compartido de VPS	Los techos reales pueden ser **hasta la mitad**. Por eso el factor 0,5
Todo corre en la misma máquina: sin latencia de red ni HTTPS	En producción cada pedido suma la ida y vuelta por internet y el cifrado
Mailpit responde al instante	Con Gmail, registro, invitaciones y avisos **frenan el pedido** que los dispara (el envío no es asíncrono). En las pruebas cada niño tiene un solo miembro, así que no hubo avisos al cargar aportes
Cada prueba se corrió una vez	Puede haber variaciones de ±10 % entre corridas
Datos homogéneos	En la realidad habrá niños con muchos más aportes que otros
No se probó el frontend en el navegador	nginx sirve archivos estáticos y no fue un factor
No se probaron invitaciones, acuerdos, eventos ni notificaciones por mail	Quedan para una segunda ronda

2.12 Cómo repetirlo

Los scripts quedaron en `~/Documents/Trama-pruebas`:

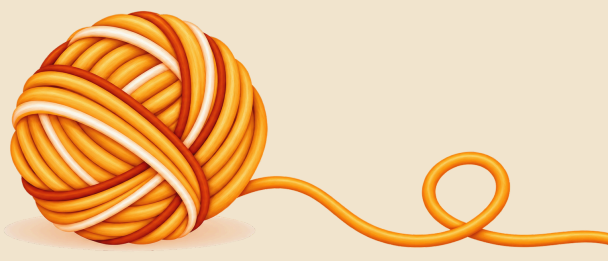
Script	Qué hace
`correr.sh preparar`	Crea las 200 familias y guarda la copia de la base
`correr.sh <servidor>`	Escalera hasta saturar (`RELOGIN=0` para la variante)
`modulos.sh`	Costo por flujo con JFR
`saturacion.sh <servidor>`	Diagnóstico, prueba larga, resiliencia y arranque
`limites.sh <servidor>`	Casos límite
`capacidad.sh`	Capacidad y disco lleno (nunca más de 35 GB)
`subidas.sh`	Reproduce la falla de archivos con JFR
`datos.js` , `graficos*.js` , `jfr-analisis.js`	Resúmenes, gráficos y análisis de JFR

TRAMA

CAPÍTULO 3

Rendimiento

El techo de cada servidor y la cadena que lo satura



3.1 Escalera hasta saturar

CPU expresada en porcentaje de **un** núcleo (backend + MySQL). En los servidores de 1 CPU el total disponible es 100 %; en el de 2 CPU, 200 %.

1 CPU · 1 GB

Con el compose actual no arranca: MySQL 8 con su configuración de fábrica necesita más de los 320 MB que le quedan y el sistema lo mata al iniciar (código de salida `137`, `OOMKilled`).

Ajustado (sin `performance_schema`, buffer de 64 MB, Java con heap al 70 %):

Usuarios	p95	Mediana	Errores	Pedidos/s	CPU	RAM backend	RAM MySQL
10	91 ms	21 ms	0 %	7	19 % + 4 %	418 MB	139 MB
25	148 ms	13 ms	0 %	18	32 % + 9 %	419 MB	158 MB
50	—	—	**95,6 %**	4	—	**424 MB (límite)**	158 MB

El backend vive pegado a su límite de memoria desde el arranque y con 50 usuarios el sistema lo mata. **No es una opción viable.**

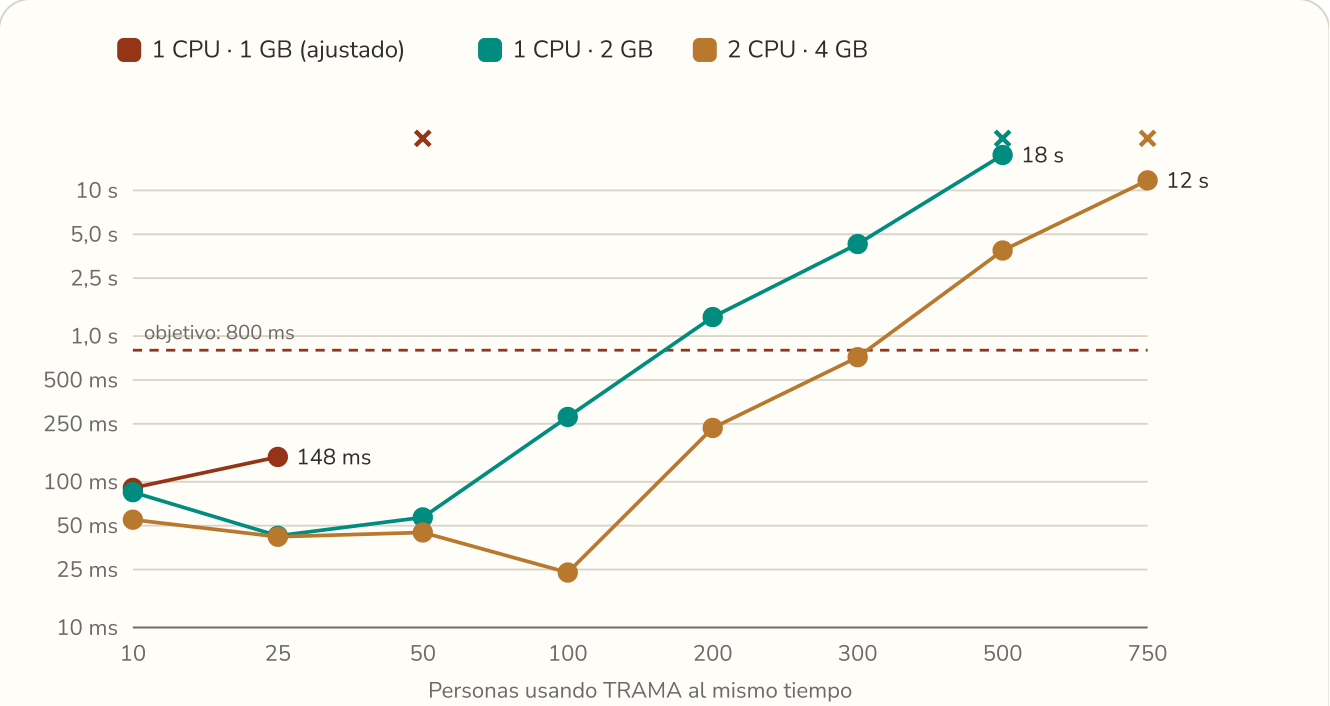
1 CPU · 2 GB

Usuarios	p95	Mediana	Errores	Pedidos/s	CPU	RAM backend	RAM MySQL
10	85 ms	21 ms	0 %	7	21 % + 3 %	423 MB	381 MB
25	43 ms	12 ms	0 %	19	25 % + 6 %	432 MB	384 MB
50	57 ms	9 ms	0 %	37	32 % + 9 %	437 MB	391 MB
100	279 ms	9 ms	0 %	71	48 % + 13 %	508 MB	401 MB
200	1,4 s	33 ms	0 %	**118**	73 % + 18 %	562 MB	413 MB
300	4,3 s	692 ms	0 %	109	77 % + 19 %	566 MB	419 MB
500	17,5 s	2,5 s	0,6 %	80	83 % + 14 %	601 MB	422 MB

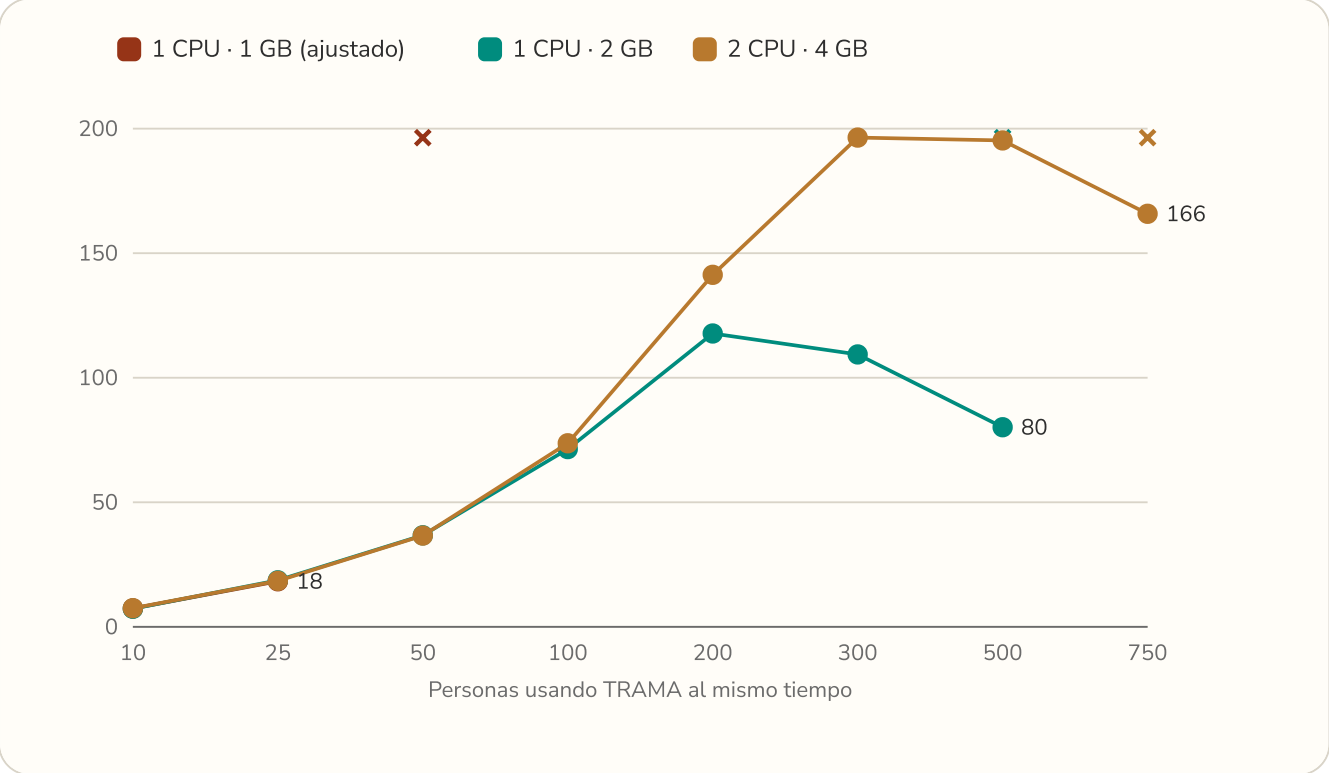
2 CPU · 4 GB

Usuarios	p95	Mediana	Errores	Pedidos/s	CPU	RAM backend	RAM MySQL
10	55 ms	18 ms	0 %	7	24 % + 5 %	504 MB	386 MB
50	45 ms	8 ms	0 %	37	37 % + 12 %	516 MB	396 MB
100	24 ms	6 ms	0 %	74	55 % + 16 %	531 MB	394 MB
200	234 ms	7 ms	0 %	141	103 % + 26 %	577 MB	407 MB
300	716 ms	14 ms	0 %	**196**	138 % + 36 %	688 MB	429 MB

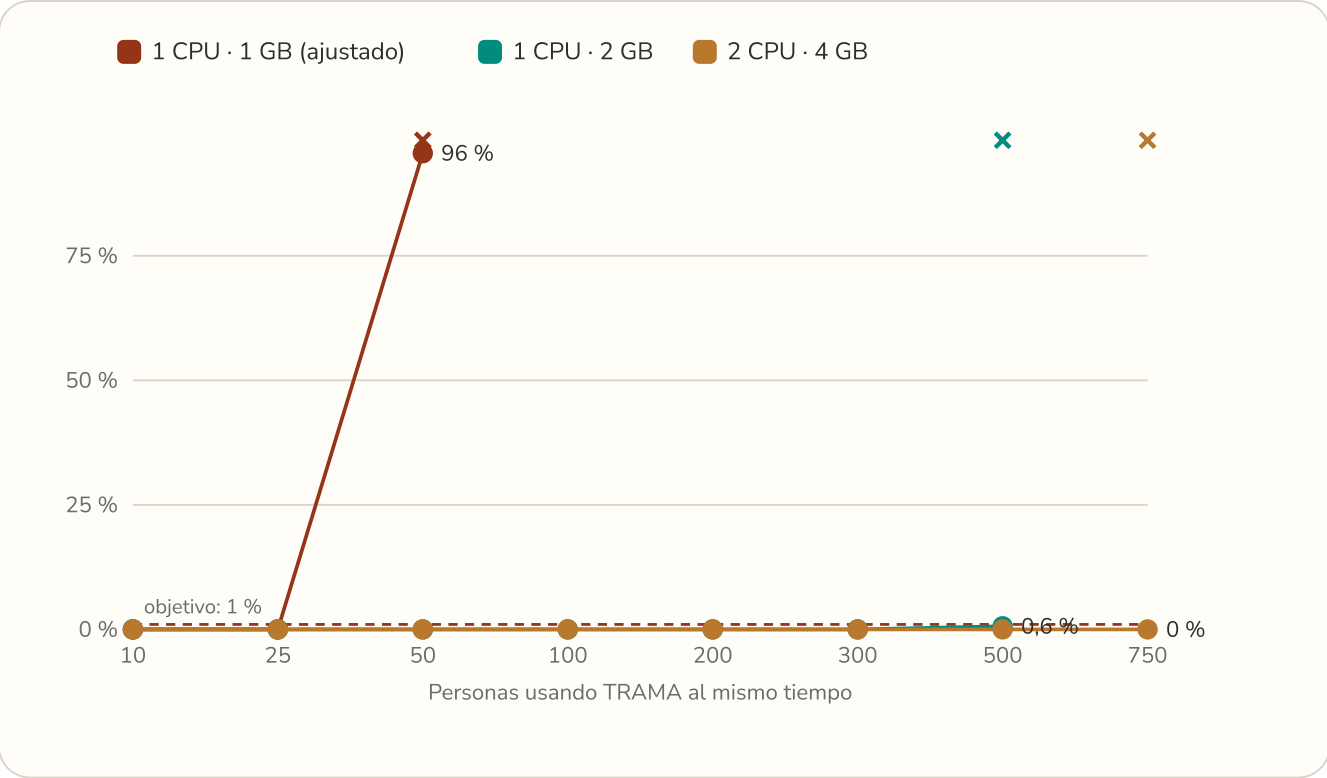
Usuarios	p95	Mediana	Errores	Pedidos/s	CPU	RAM backend	RAM MySQL
500	3,9 s	701 ms	0 %	195	152 % + 40 %	691 MB	431 MB
750	11,7 s	2,0 s	0 %	166	155 % + 38 %	693 MB	435 MB



Tiempo de respuesta p95



Pedidos atendidos por segundo



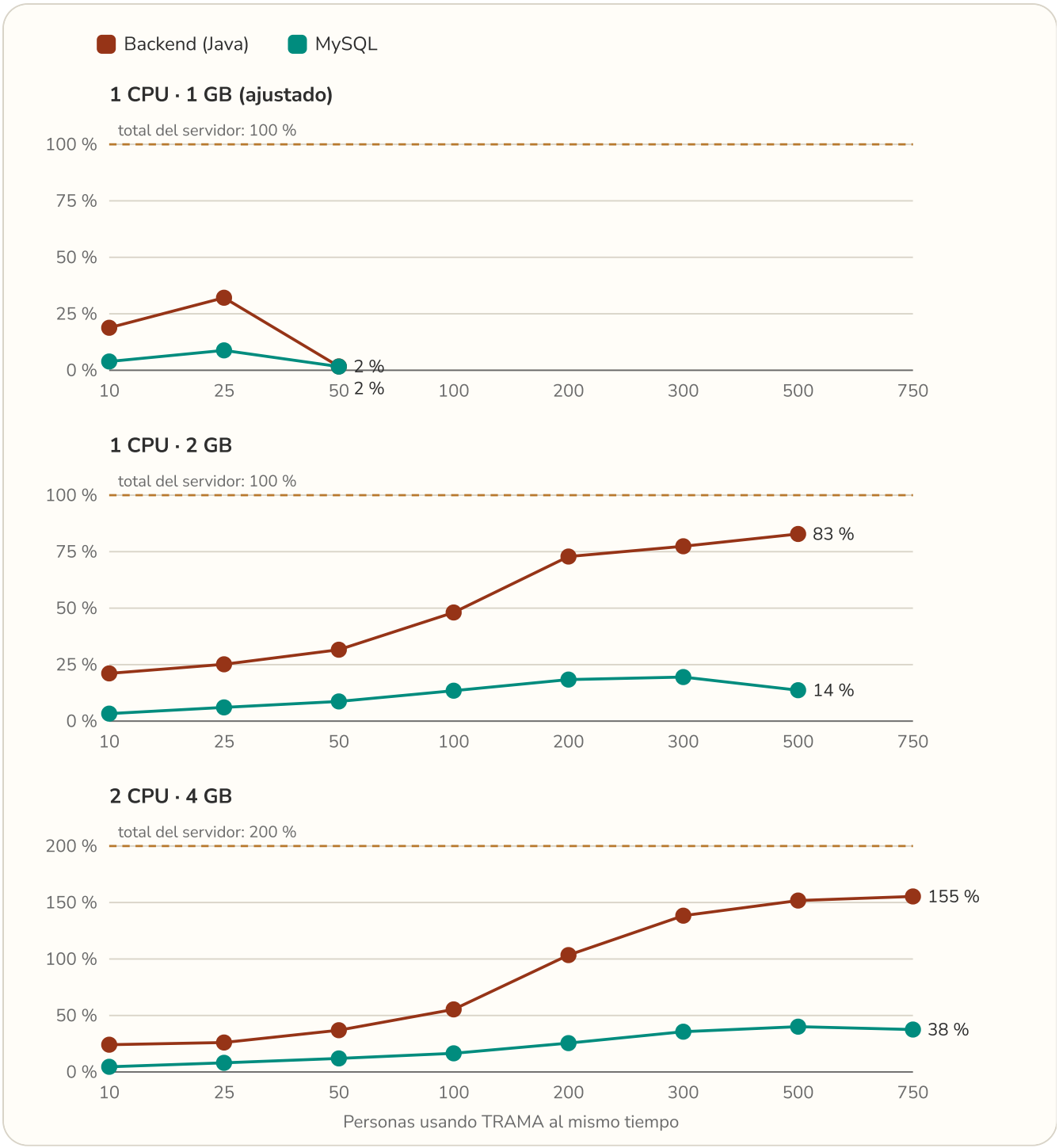
Pedidos con error

Lo que muestran las tablas

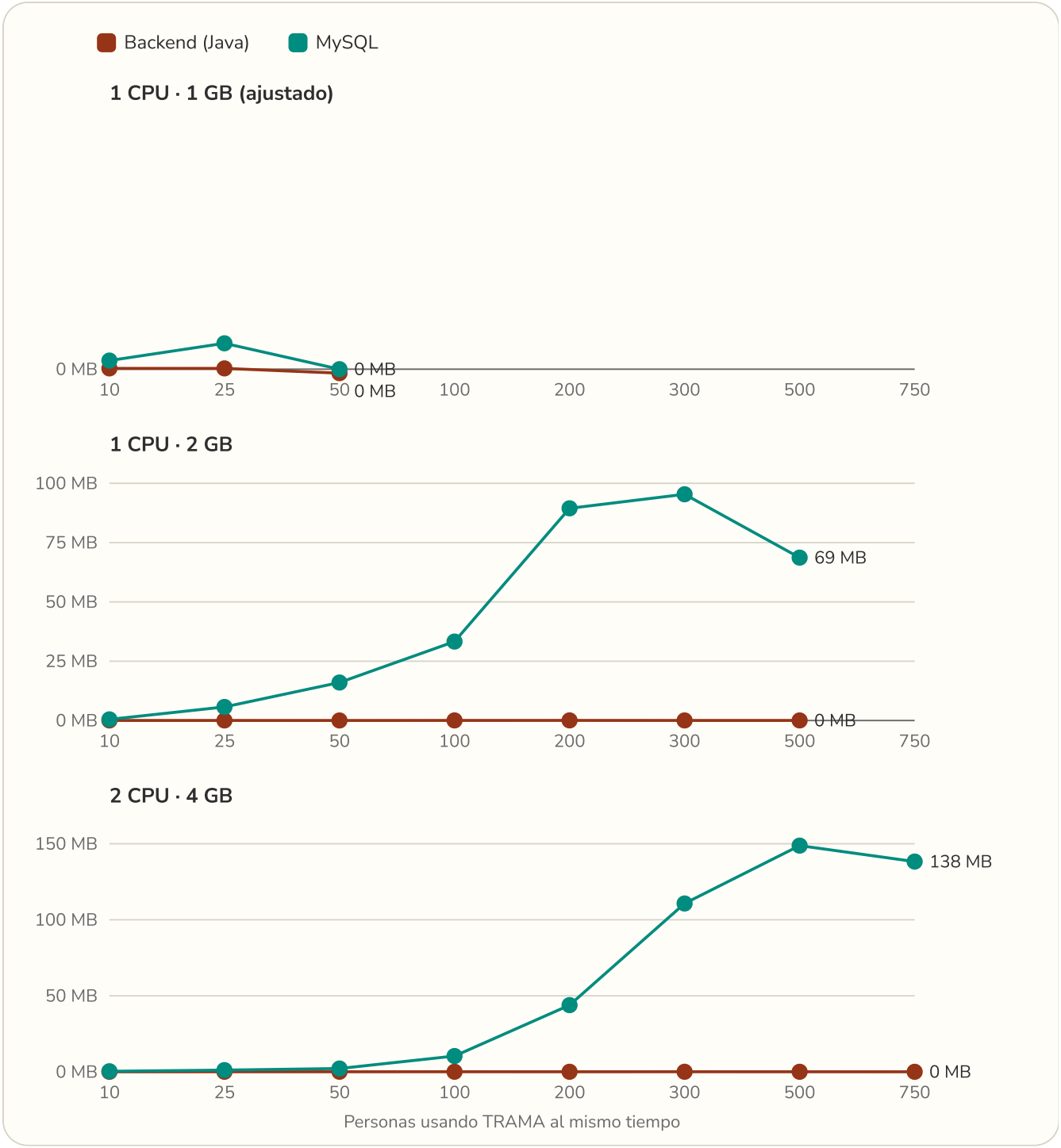
- **La memoria nunca fue el límite** en 2 GB y 4 GB: el backend usó como máximo 601 MB de 1.128 y 693 MB de 2.416. El límite siempre es la **CPU**.
- **Pasado el techo, el servidor atiende menos, no más.** En 1 CPU · 2 GB, de 118 pedidos/s con 200 usuarios baja a 80 con 500: gasta más tiempo repartiéndose entre pedidos que atendiéndolos.
- **La mediana se mantiene baja mucho después de que el p95 se dispara.** La mayoría de los pedidos sigue rápida y unos pocos esperan muchísimo: es el síntoma de una **cola**, no de un sistema parejamente lento (ver la cadena de saturación).



Memoria por escalón



CPU por escalón



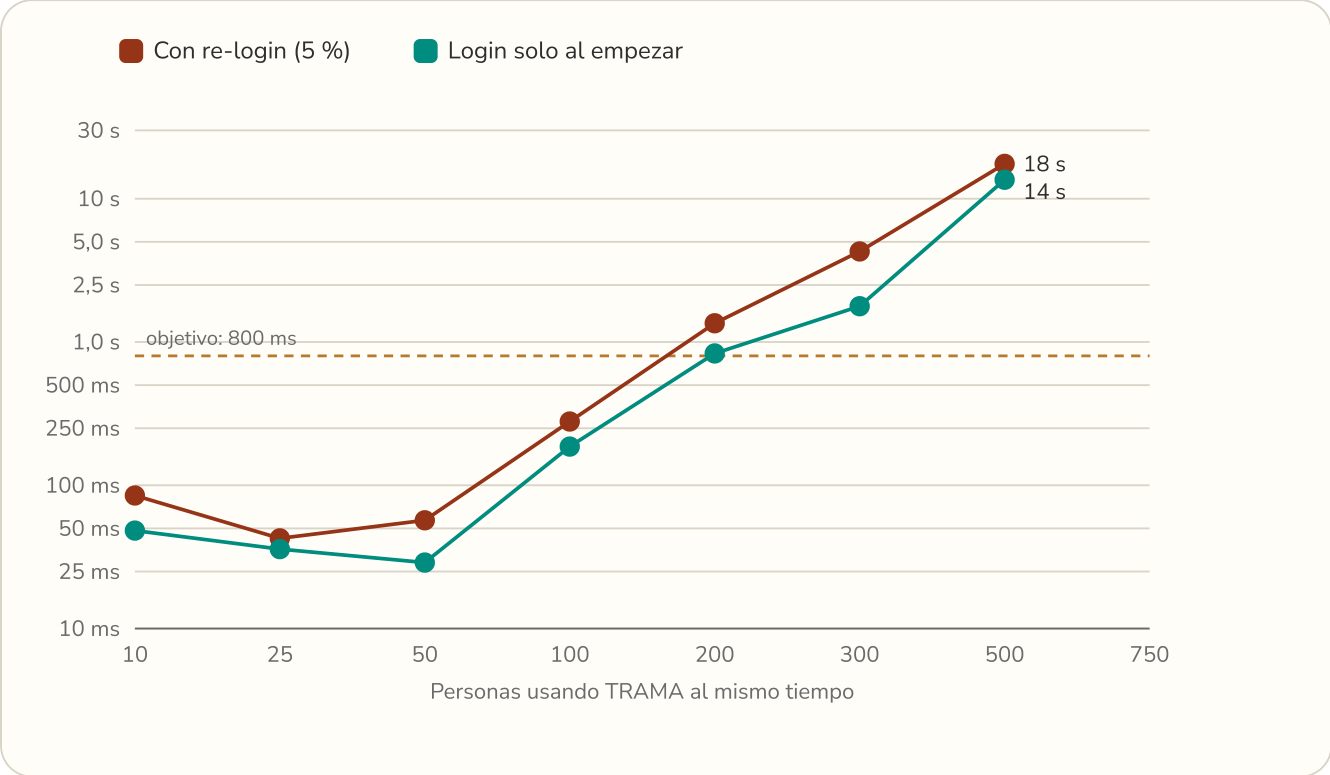
Disco escrito por MySQL en cada escalón

3.2 Variante "login una vez"

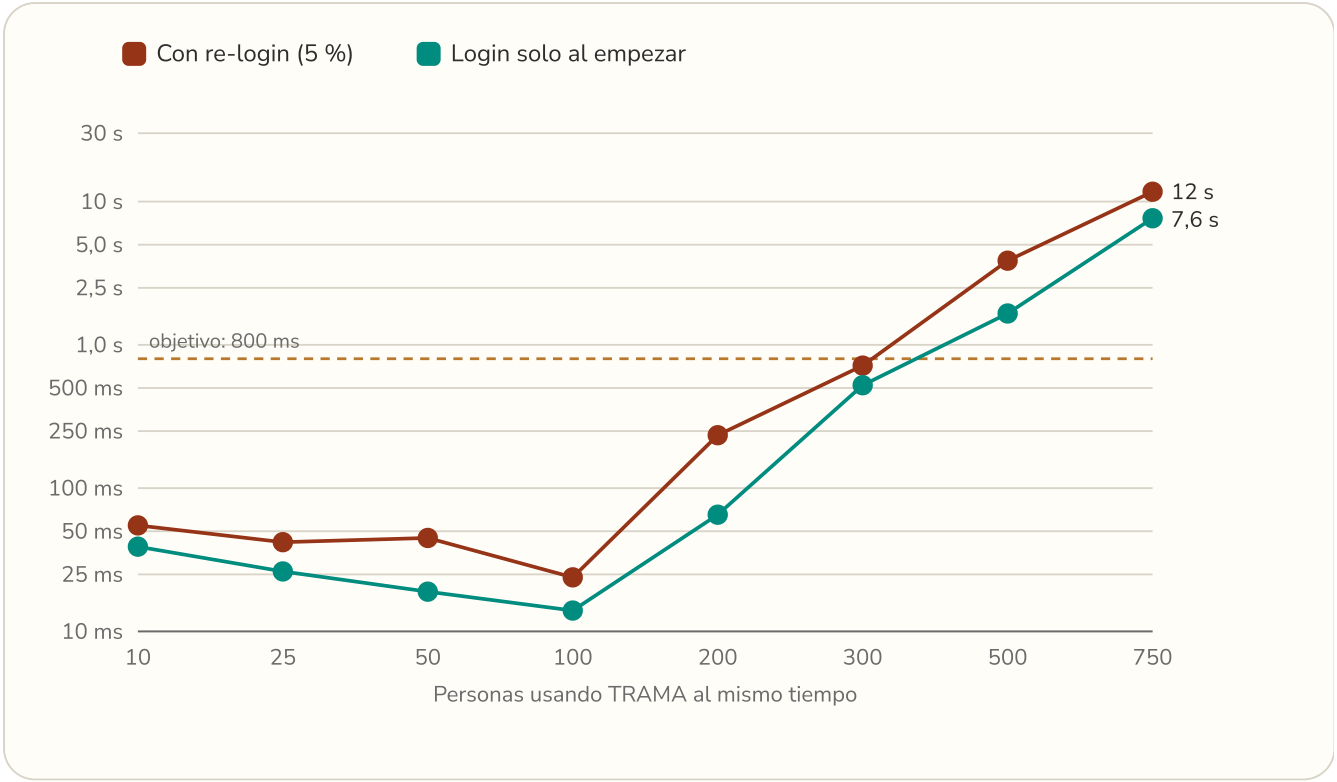
Usuarios	2 GB con re-login	2 GB login una vez	4 GB con re-login	4 GB login una vez
100	279 ms · 71/s	186 ms · 70/s	24 ms · 74/s	14 ms · 74/s
200	1,4 s · 118/s	832 ms · 128/s	234 ms · 141/s	65 ms · 146/s
300	4,3 s · 109/s	1,8 s · **162/s**	716 ms · 196/s	523 ms · 202/s

Usuarios	2 GB con re-login	2 GB login una vez	4 GB con re-login	4 GB login una vez
500	17,5 s · 80/s	13,6 s · 117/s	3,9 s · 195/s	1,7 s · **282/s**
750	—	—	11,7 s · 166/s	7,6 s · 242/s

Sacando los re-logins el techo sube un **37 %** en 2 GB y un **44 %** en 4 GB. Aun así, cada escalón empieza con todos los usuarios entrando en los mismos 10 segundos: eso es lo que pasaría **a la hora de entrada a la escuela**, y es lo que termina saturando.



Peso de los logins en 1 CPU · 2 GB

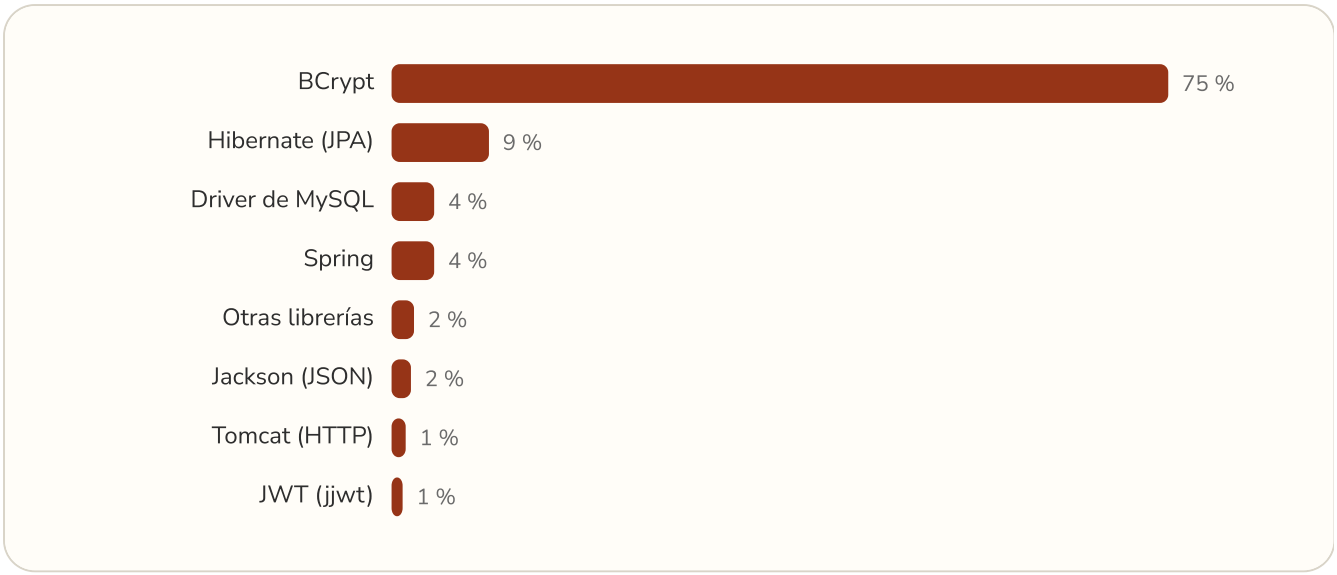


Peso de los logins en 2 CPU · 4 GB

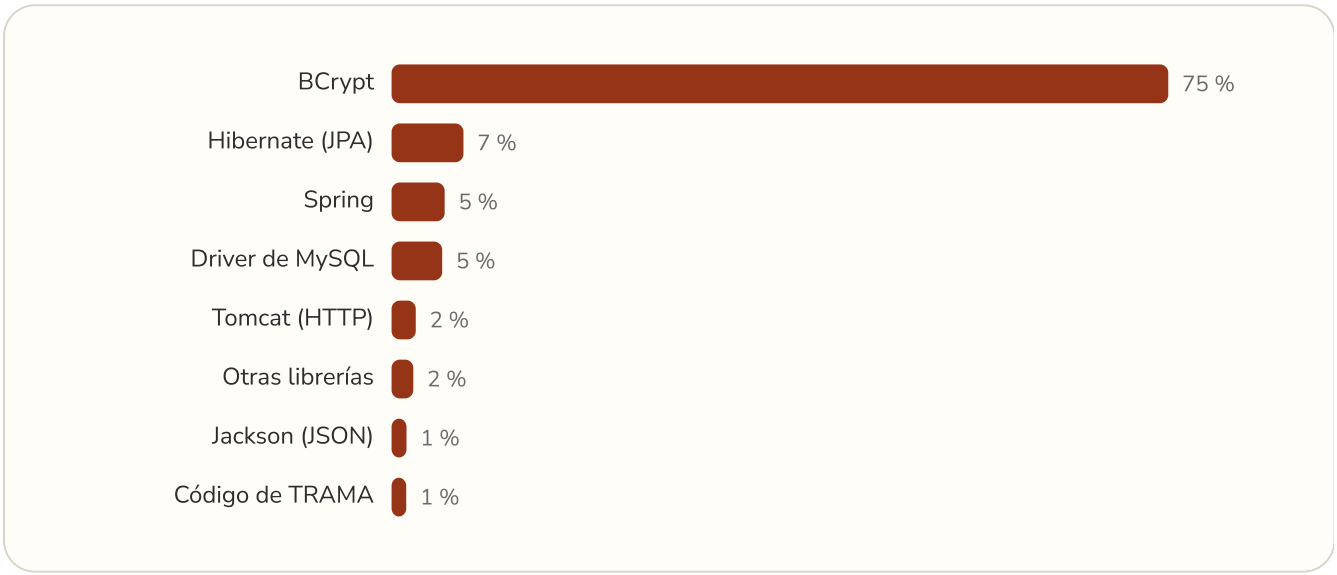
3.3 Qué satura a TRAMA

Grabaciones de Java Flight Recorder durante un minuto de saturación:

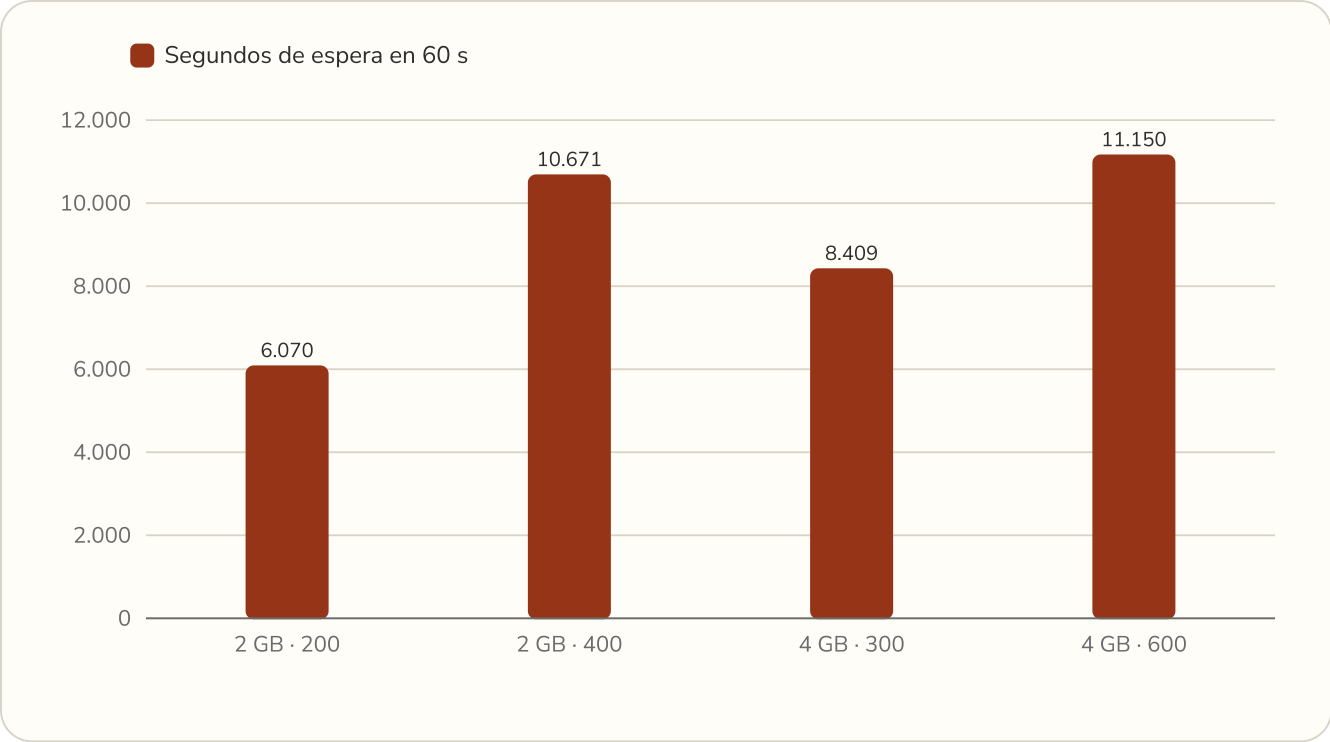
Medición	2 GB · 200 usuarios	2 GB · 400	4 GB · 300	4 GB · 600
CPU en **BCrypt**	57 %	**75 %**	78 %	75 %
CPU en Hibernate + driver MySQL	23 %	13 %	10 %	12 %
Veces que un pedido esperó una conexión	7.921	6.527	18.065	15.633
Tiempo total esperando conexión	6.070 s	**10.671 s**	8.409 s	11.150 s
Espera más larga	1,4 s	5,8 s	0,8 s	1,1 s
Pausa más larga del recolector de basura	174 ms	159 ms	25 ms	27 ms



CPU por capa con el servidor saturado (2 GB)



CPU por capa con el servidor saturado (4 GB)



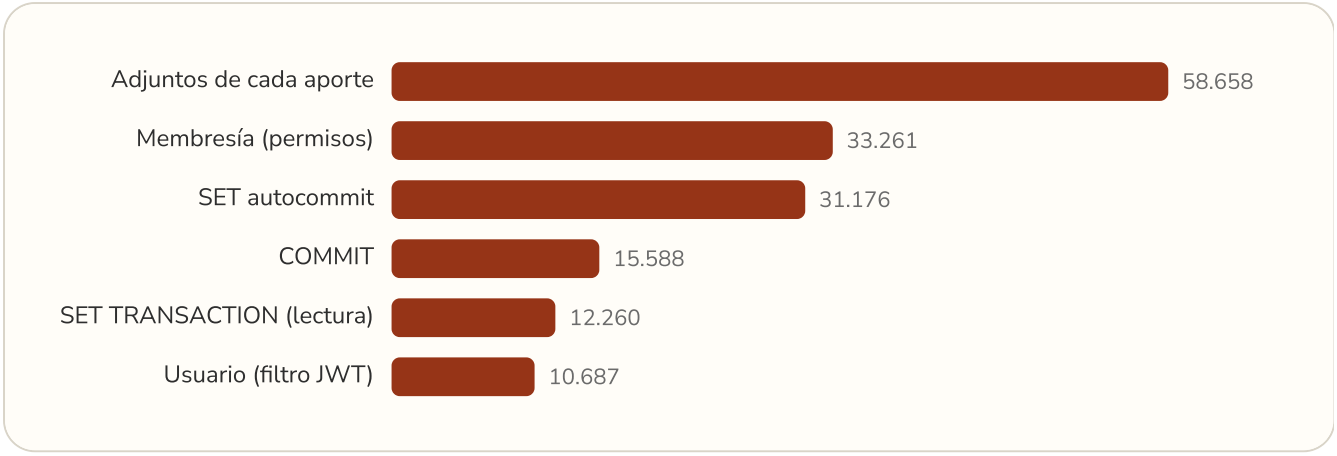
Segundos esperando una conexión a la base

La cadena:

1. **BCrypt** cuesta alrededor de 100 ms de CPU por login, a propósito: es lo que hace difícil adivinar contraseñas. Con muchos logins se come la CPU.
2. TRAMA no configura `spring.jpa.open-in-view`, así que queda **activado** (el propio backend lo avisa al arrancar). Con eso, `AuthenticationService.authenticate` busca al usuario, **toma una conexión y no la suelta hasta terminar el pedido**, incluido el cálculo de BCrypt.
3. Hikari tiene **10 conexiones** (valor por defecto). Con la CPU saturada, BCrypt tarda más y las conexiones quedan tomadas más tiempo. **Todos** los pedidos —no solo los logins— esperan una conexión libre. En 60 segundos se acumularon más de 10.000 segundos de espera: en promedio, unos 170 pedidos parados a la vez.
4. Cuando la espera pasa los 30 s, Hikari corta con `Connection is not available, request timed out after 30000ms` y la persona recibe `500`.

Factores que suman:

- **Filtro JWT en cada pedido.** `JwtAuthenticationFilter` parsea y verifica la firma del token tres veces (`extractUsuarioId`, `isTokenValid` → `extractUsuarioId` e `isTokenExpired`) y busca al usuario en la base. Es entre el 11 % y el 24 % de la CPU de cada pedido.
- **Unas 27 consultas SQL por pedido.** Con 200 usuarios en 2 GB hubo 202.345 consultas para 7.353 pedidos. Las principales: adjuntos de cada aporte (58.658 veces, problema N+1), membresía del usuario (33.261 veces: los permisos se verifican unas 4,5 veces por pedido) y el usuario del filtro JWT (10.687).
- **Recolector de basura Serial en 1 CPU.** Con menos de 2 CPU o menos de ~1,8 GB, Java elige el recolector *Serial*, que detiene la aplicación entera mientras limpia: pausas de hasta 174 ms. Con 2 CPU usa G1 y las pausas bajan a 27 ms.



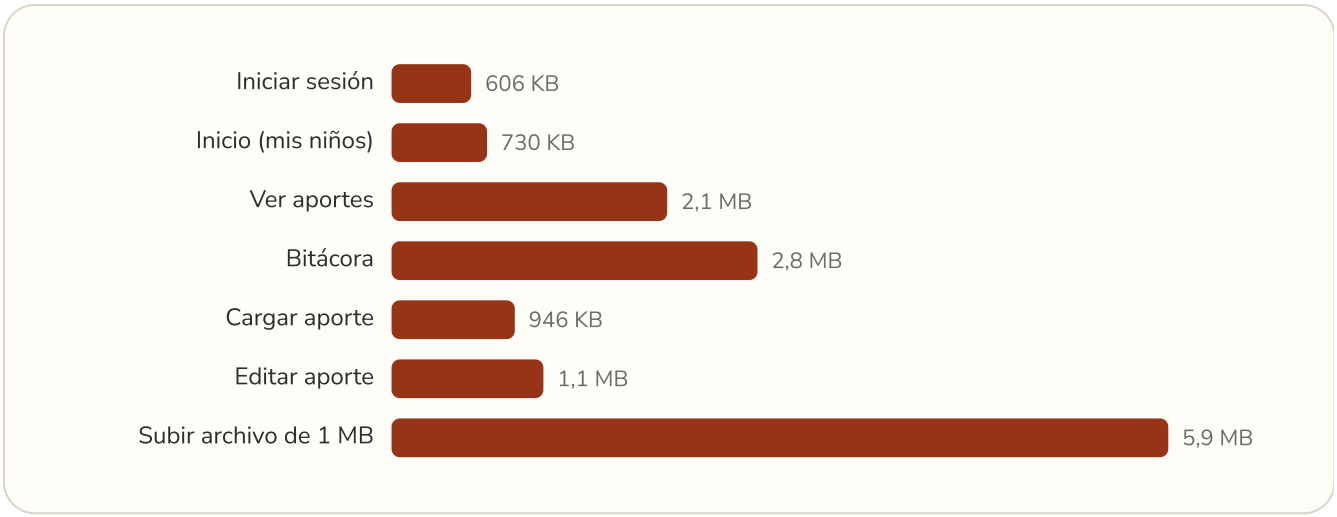
Las consultas que más se repiten

MySQL no es el cuello de botella. Ninguna consulta tardó en promedio más de 0,2 ms, el 99,9 % de las lecturas salió de memoria, no hubo esperas por bloqueos de filas ni consultas lentas. El costo está del lado de Java, en hacer tantos viajes.

3.4 Costo de cada flujo

Cada flujo solo, sin saturar, en 1 CPU · 2 GB:

Flujo	p95	Memoria pedida por pedido	Qué usa la CPU
Iniciar sesión	117 ms	606 KB	**BCrypt 98 %**
Inicio (mis niños)	23 ms	730 KB	Filtro JWT 24 %, Tomcat 23 %, proyectos 21 %
Ver aportes (20 aportes)	29 ms	2,1 MB	Aportes 40 %, permisos 13 %, filtro JWT 12 %
Bitácora	33 ms	2,8 MB	Aportes 40 %, permisos 12 %, filtro JWT 11 %
Cargar aporte	27 ms	946 KB	Filtro JWT 24 %, Tomcat 21 %, aportes 16 %
Editar aporte	31 ms	1,2 MB	Tomcat 28 %, aportes 24 %, filtro JWT 17 %
Subir archivo de 1 MB	97 ms	6 MB	Tomcat 69 % (leer el archivo), permisos 8 %



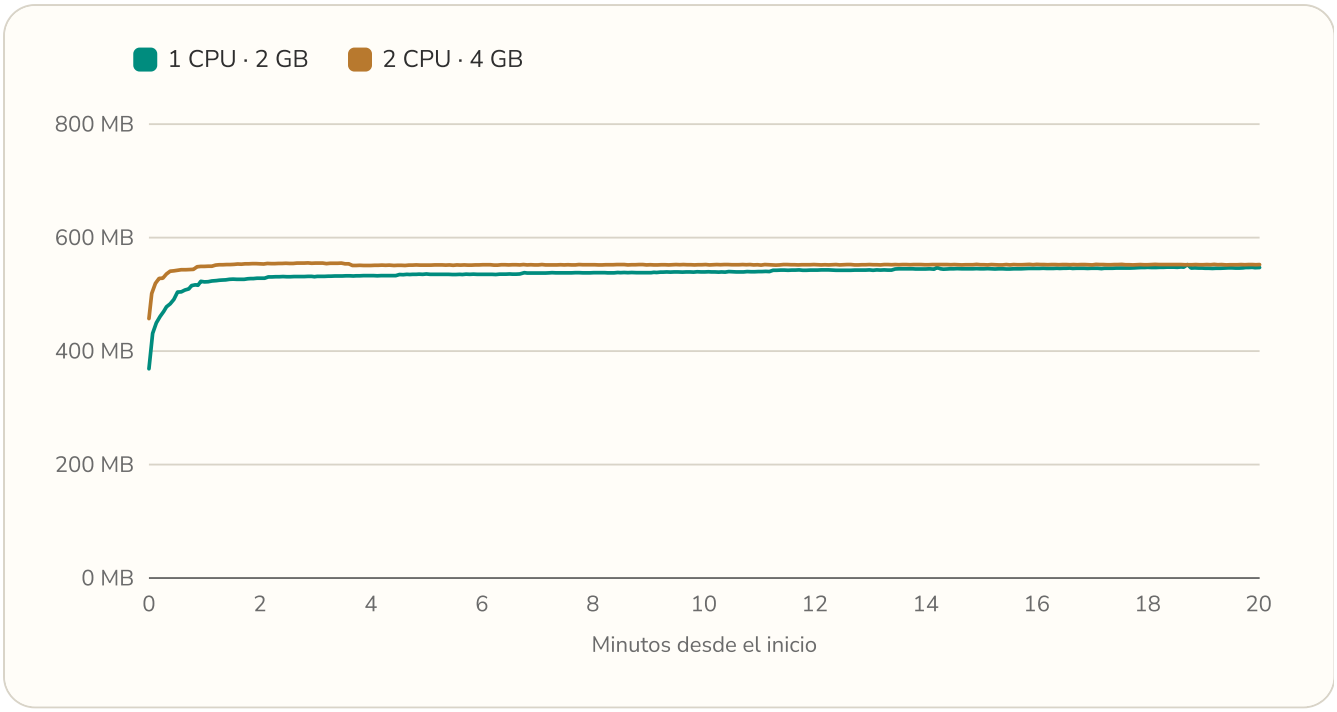
Memoria por pedido y flujo

Por qué cada uno pesa lo que pesa:

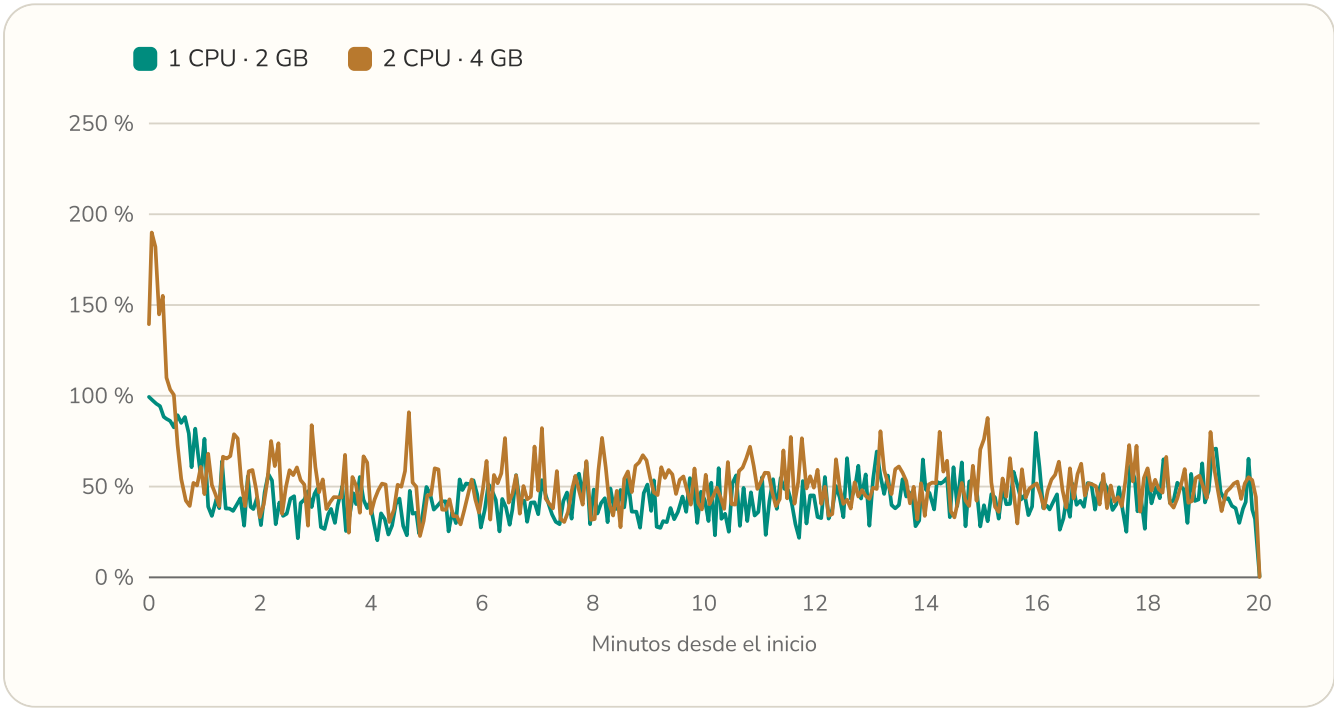
- **Ver aportes y bitácora** son los más pesados en memoria aunque el niño tenga solo 20 aportes: por cada aporte se descifra el contenido y los datos, se hace una consulta de adjuntos y se arma la respuesta JSON completa. Ese costo crece **en proporción a la cantidad de aportes**, y por eso es el límite real (ver capítulo 4 (Casos límite)).
- **Subir un archivo** pide unas 6 veces su tamaño: Tomcat lo recibe, `getBytes()` lo copia, `cifrar` hace otra copia y escribir en disco usa un buffer más.
- **El filtro JWT** aparece en todos: es un costo fijo por pedido que pesa más en los flujos livianos (inicio, cargar).
- Toda esta memoria es temporal. El heap usado nunca pasó de 88 MB en estos flujos; lo que crece es el trabajo del recolector de basura.

3.5 Prueba larga (20 minutos, 100 usuarios)

Servidor	p95	Errores	Pedidos	RAM del backend
1 CPU · 2 GB	87 ms	0 %	90.556	de 528 a 553 MB (+25 MB)
2 CPU · 4 GB	33 ms	0 %	92.027	estable en 553 MB



Memoria del backend durante 20 minutos

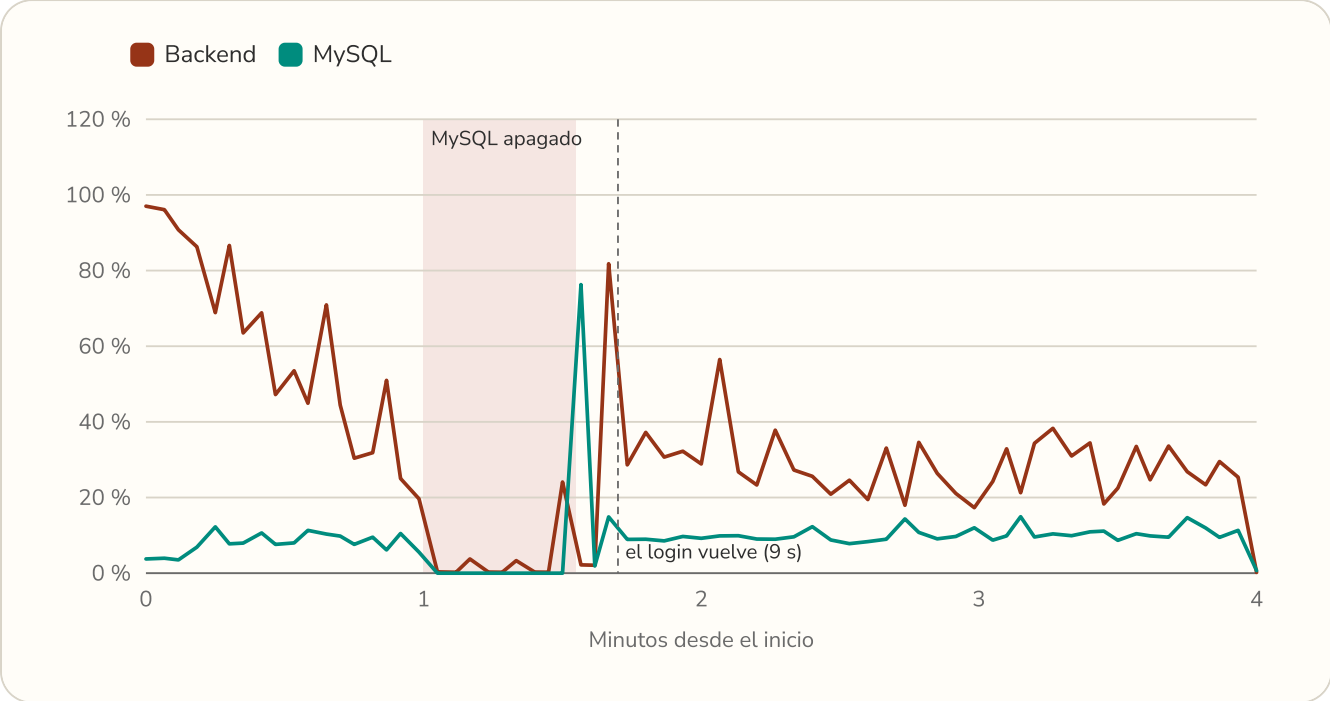


CPU del backend durante 20 minutos

No se detectó fuga de memoria. La leve suba en 2 GB coincide con los aportes que se van cargando durante la prueba, que hacen cada lista un poco más grande. Antes de producción conviene una prueba de varias horas.

3.6 Resiliencia: MySQL se cae con gente usando la app

Servidor	MySQL apagado	El login vuelve a funcionar	Errores (de ~7.600 pedidos)
1 CPU · 2 GB	33 s	9 s después de prender MySQL	0,8 %
2 CPU · 4 GB	34 s	8 s después de prender MySQL	0,8 %



Resiliencia: MySQL apagado con usuarios activos

TRAMA **se recupera sola**, sin reiniciar el backend. Hubo pocos errores porque, sin base, los pedidos **se quedan esperando** una conexión (hasta 30 s) en lugar de fallar al instante: para la persona, la app parece colgada durante el corte. Los errores fueron `Communications link failure` en las conexiones abiertas justo en el momento del corte.

3.7 Arranque

Servidor	Tiempo hasta responder	RAM del backend sin uso	RAM de MySQL sin uso
1 CPU · 2 GB	11 s	350 MB	374 MB
2 CPU · 4 GB	8 s	573 MB	381 MB

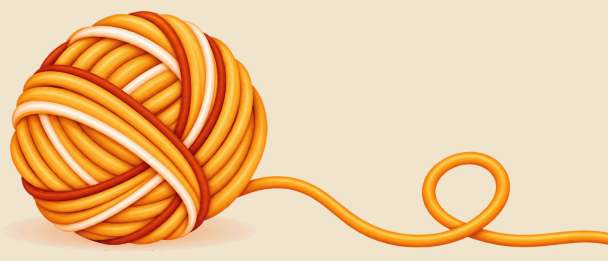
Java toma un tamaño de heap proporcional al límite del contenedor (25 % por defecto), por eso el backend ocupa más con más memoria disponible.

TRAMA

CAPÍTULO 4

Casos límite

Muchos aportes, ediciones simultáneas, archivos, logins y disco lleno

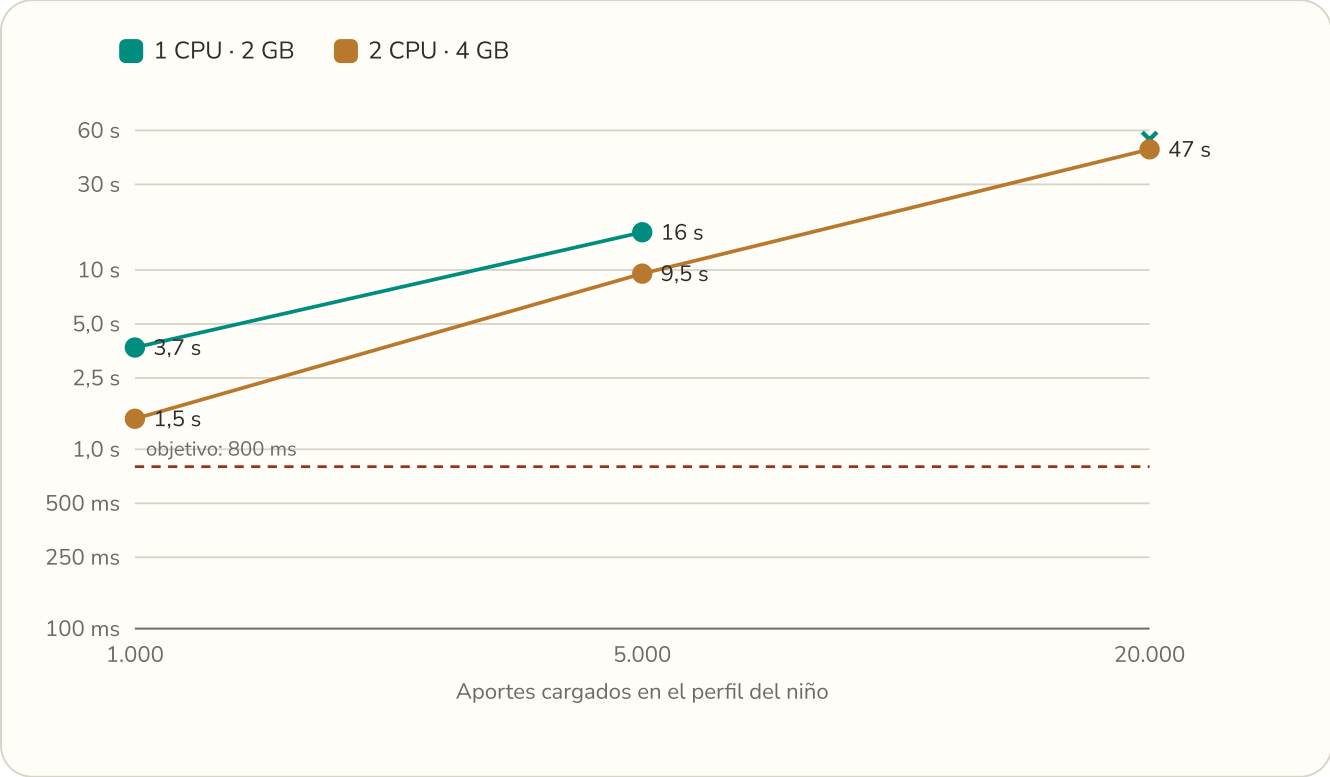


Cada caso se corrió en **1 CPU · 2 GB** y **2 CPU · 4 GB**. Las respuestas se contaron por código HTTP, para saber exactamente qué recibe la persona.

4.1 Niños con muchos aportes

20 personas mirando a la vez el perfil (lista de aportes + bitácora) del mismo niño:

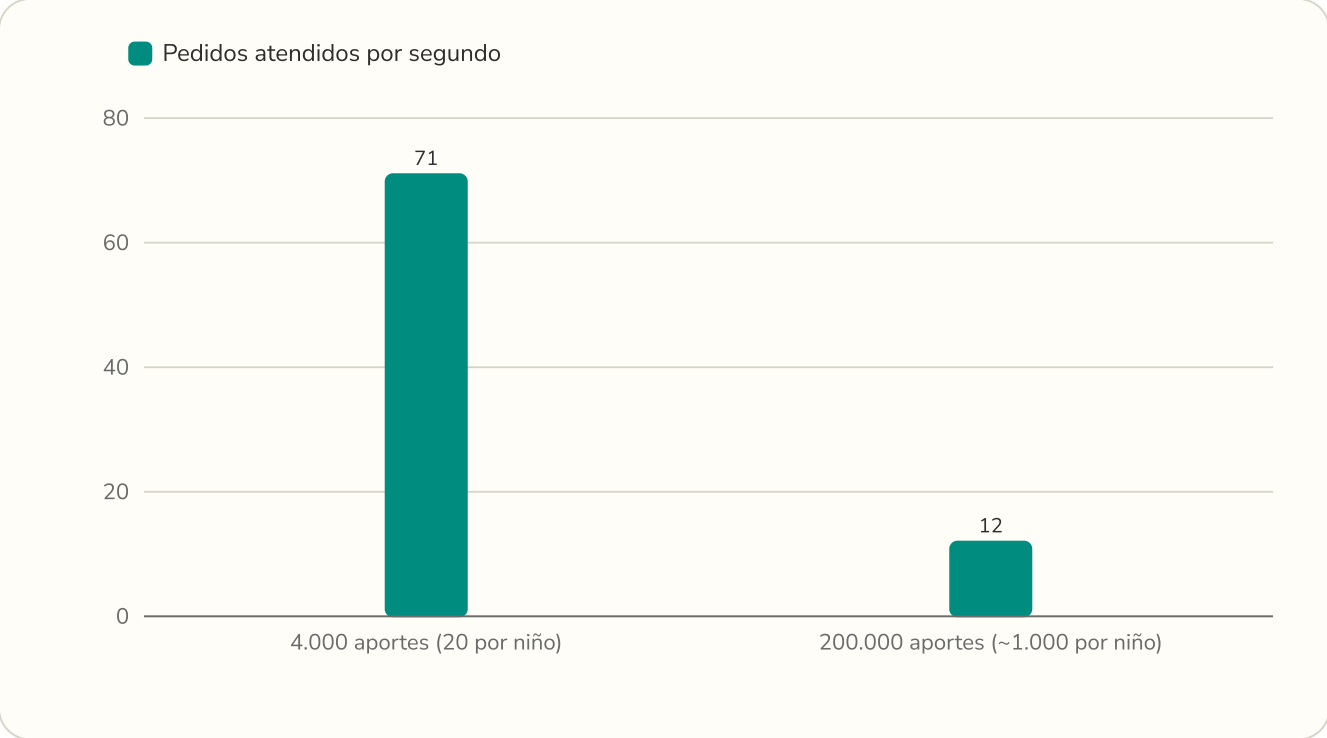
Aportes del niño	2 GB · p95	2 GB · mediana	4 GB · p95	Errores
1.000	**3,7 s**	2,0 s	1,5 s	0 %
5.000	**16,2 s**	9,6 s	9,5 s	0 %
20.000	**60 s (corte)**	45 s	47 s	2 GB: **91 %, el backend dejó de responder** · 4 GB: 0 %



Abrir el perfil según la cantidad de aportes

Y con la base grande completa (200.000 aportes, unos 1.000 por niño), la misma carga mezclada de siempre en 1 CPU · 2 GB:

100 usuarios	4.000 aportes en la base	200.000 aportes en la base
p95 general	279 ms	**15,3 s**
p95 de ver el perfil	50 ms	**14,8 s**
Pedidos atendidos por segundo	71	**12**



Misma carga con base chica y base grande

Por qué pasa. `GET /api/proyectos/{id}/aportes` y `GET /bitacora` devuelven **todos** los aportes del niño en cada pedido:

1. Se traen todas las filas de `aporte` del niño.
2. Por **cada** aporte se hace una consulta para buscar sus adjuntos (problema N+1): con 20.000 aportes son 20.000 consultas para abrir un perfil.
3. Por cada aporte se descifran `contenido` y `datos` (AES).
4. Se arma un JSON con todo, que el navegador tiene que bajar y dibujar.

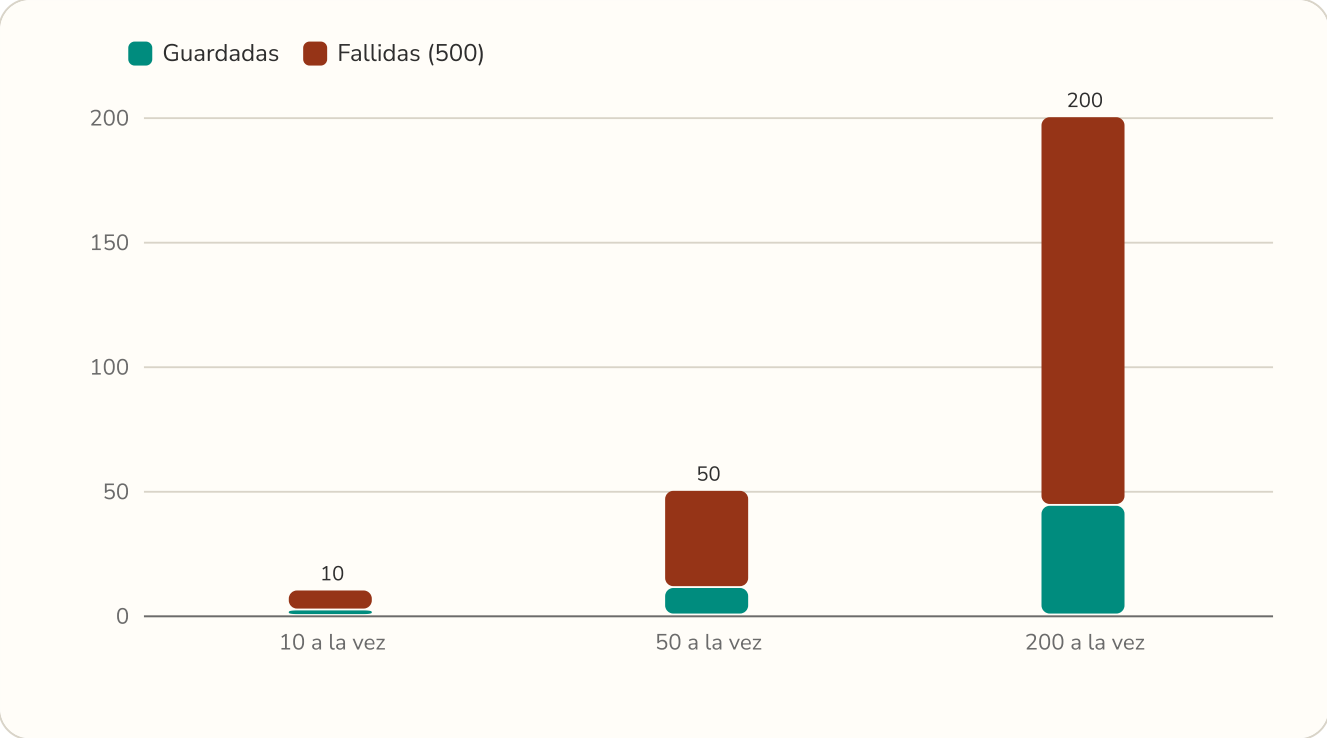
El tamaño total de la base no importa: MySQL respondió rápido aun con 200.000 aportes. **Importa cuántos aportes tiene cada niño.** Un niño con 4 o 5 personas que cargan un par de cosas por día llega a 1.000 aportes en menos de un año.

Qué haría: paginar las dos listas (por ejemplo, de a 50, ordenadas por fecha), traer los adjuntos de la página con **una** consulta (`IN (...)` o `JOIN FETCH`) y, si hace falta, un índice por `(proyecto_id, creado_en)`.

4.2 Ediciones simultáneas

Varias personas guardando **el mismo aporte** al mismo tiempo (en la práctica: el autor con la app abierta en dos dispositivos, o un doble clic en "Guardar"):

Ediciones a la vez	Guardadas (`200`)	Fallidas (`500`)	Versiones en la base
10	2	8	2
50	11 a 13	37 a 39	11 a 13
200	41 a 44	156 a 159	41 a 44



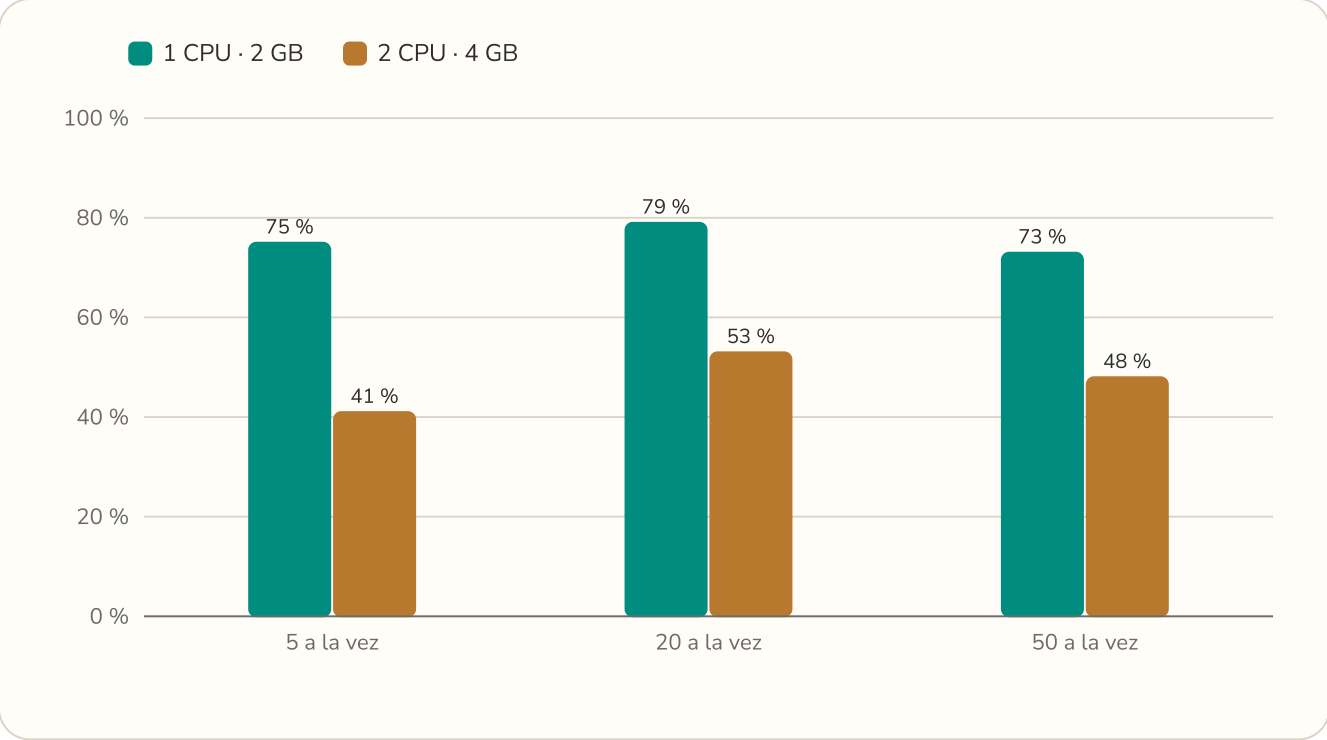
Ediciones simultáneas del mismo aporte

- **No se pierden ni se mezclan datos:** cada edición exitosa dejó exactamente una versión en `aporte_version`.
- **Pero la persona recibe `500 Ocurrió un error inesperado`.** El backend registra `Deadlock found when trying to get lock` (MySQL 1213) y `OptimisticLockException`: dos transacciones se bloquean entre sí al actualizar el aporte e insertar la versión.

Qué haría: atrapar el conflicto y responder `409` con un mensaje claro ("Este aporte se acaba de guardar desde otro lado. Revisá los cambios y volvé a intentar"), o reintentar una vez.

4.3 Archivos adjuntos

Subidas de 5 MB a la vez	2 GB: subidas fallidas	4 GB: subidas fallidas
5	75 %	41 %
20	79 %	53 %
50	73 %	48 %



Subidas que fallan después de un pico de uso

Con un backend **recién iniciado**, 5 personas subiendo y bajando archivos de 1 MB y de 5 MB: **0 fallas** en 242 subidas y 242 descargas. Las fallas aparecen **después de un pico de uso**.

Causa (confirmada con Java Flight Recorder): 806 `OutOfMemoryError` de **memoria directa** (`java.nio.Bits.reserveMemory`), 622 al subir y 184 al bajar:

1. `ArchivoService.subir` (línea 142) y `ArchivoService.descargar` (línea 211) usan `Files.write` y `Files.readAllBytes`, que trabajan con el archivo **entero** en memoria.
2. Por debajo, Java copia los bytes a un buffer de memoria directa **del tamaño del archivo** y lo **guarda en cada hilo** para reutilizarlo.
3. Tomcat tiene hasta 200 hilos. Después de un pico (en la prueba, la edición simultánea con 200 personas), muchos hilos guardan su buffer de 5 MB. Cuando la suma pasa el límite de memoria directa (igual al heap: ~280 MB en 2 GB y ~600 MB en 4 GB), **toda subida o descarga nueva falla**.
4. `GlobalExceptionHandler` lo devuelve como `500` **sin escribirlo en el log**, así que en producción no habría rastro.

En producción aparecería después del primer pico de uso y **no se iría hasta reiniciar el backend**.

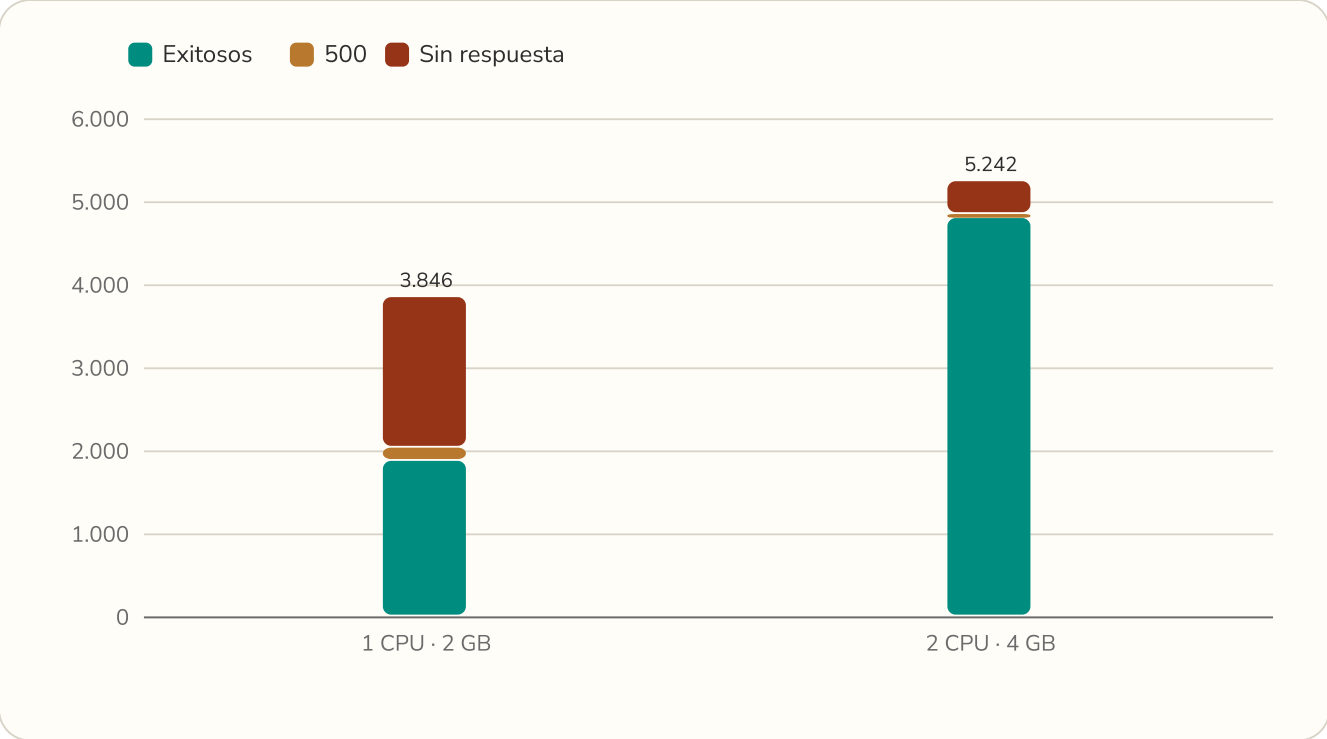
Qué haría: leer, cifrar y escribir los archivos por partes (`InputStream` → `CipherOutputStream` → archivo), y servir las descargas como *stream*. Como paliativo inmediato, sin tocar código: agregar `-Djdk.nio.maxCachedBufferSize=262144` a las opciones de Java.

4.4 Tormenta de logins

De 1 a 80 logins por segundo en 3 minutos:

Servidor	Logins exitosos	`500` (sin conexión libre)	Sin respuesta en 60 s	Promedio atendido
1 CPU · 2 GB	1.874	159	**1.813**	7,8 por segundo

Servidor	Logins exitosos	`500` (sin conexión libre)	Sin respuesta en 60 s	Promedio atendido
2 CPU · 4 GB	4.798	49	395	20 por segundo



Tormenta de logins

Cada login cuesta ~100 ms de CPU por BCrypt **y retiene una conexión a la base durante ese tiempo** (ver capítulo 3 (Rendimiento)). En las dos pruebas el backend quedó sin responder al terminar y hubo que reiniciarlo para seguir.

Esto es lo que pasaría con un **ataque de fuerza bruta** o, sin mala intención, **a la hora de entrada a la escuela** si muchas personas entran juntas. Hoy no hay un límite de intentos por IP (ver capítulo 6 (Seguridad)).

4.5 Textos largos

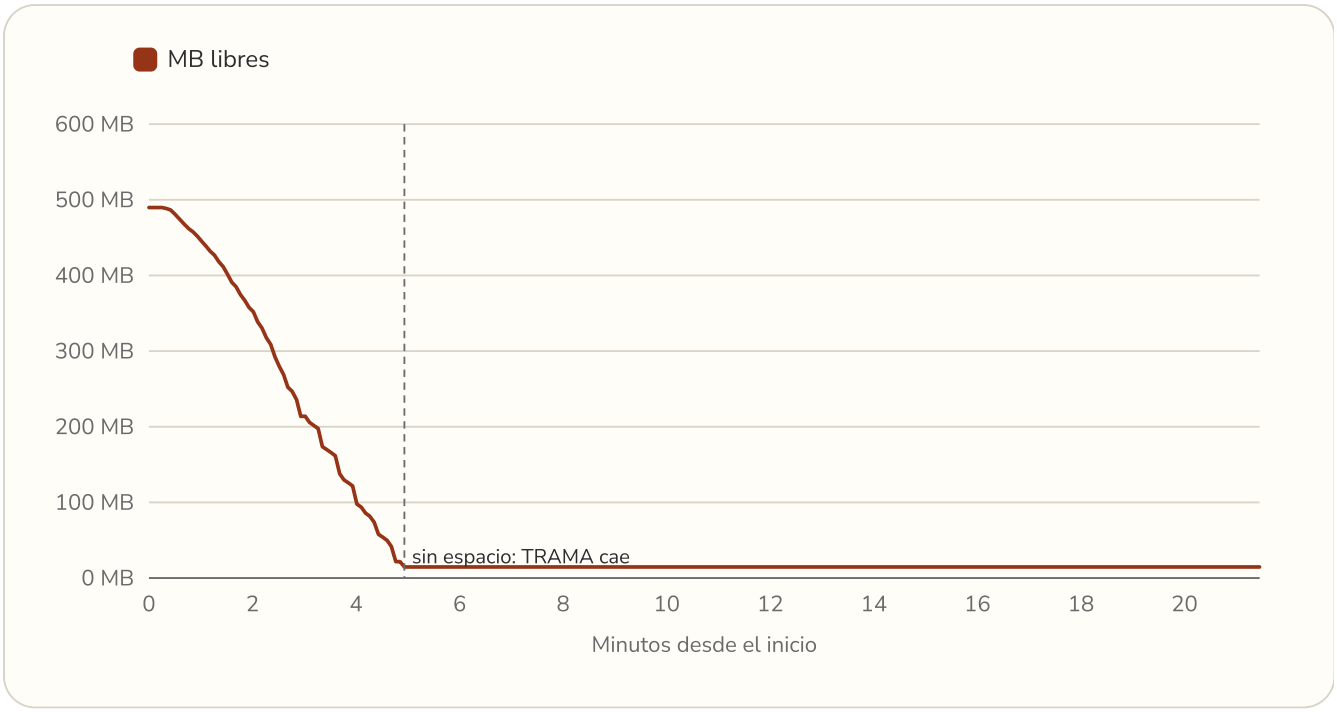
50 personas cargando aportes de 10.000 caracteres (el máximo permitido):

Servidor	Aportes guardados	p95	Errores
1 CPU · 2 GB	2.675	172 ms	0 %
2 CPU · 4 GB	2.833	42 ms	0 %

Sin problemas: el límite de 10.000 caracteres está bien elegido.

4.6 Disco lleno

MySQL en un disco de 700 MB que se llena cargando aportes. **Es el caso más grave para la operación:**



Espacio libre mientras se llena el disco

1. **Fallan las escrituras.** MySQL no puede agrandar las tablas: `The table 'aporte' is full` (8.915 veces) y `The table 'registro_accion' is full` (17.608 veces; cada carga, edición o confirmación deja una fila de registro).
2. **Falla la primera vista del día.** `registrarVista` guarda una fila por persona, niño y día; con el disco lleno esa escritura falla y el perfil da error aunque solo se quiera mirar.
3. **Se cuelga todo.** MySQL no puede escribir el binlog y queda esperando: `Disk is full writing './binlog.000003' ... Waiting for someone to free space... Retry in 60 secs`. Las 10 conexiones quedan tomadas y **el backend deja de responder por completo**, incluido el login.
4. **No se recupera sola.** Se intentó liberar espacio con `PURGE BINARY LOGS`, pero el binlog que ocupa el lugar es el que está en uso. Hace falta agrandar el disco o reiniciar MySQL a mano.

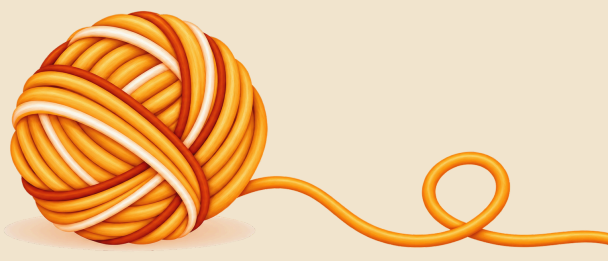
Qué haría: una alerta cuando el disco pase el 80 %, rotación de backups, bajar la retención de binlogs (`binlog_expire_logs_seconds`, hoy 30 días) o apagarlos si no hay réplica, y dejar siempre un margen libre. Ver capítulo 5 (Capacidad en 50 GB).

TRAMA

CAPÍTULO 5

Capacidad en 50 GB

Cuánto ocupa cada cosa y cuánto entra

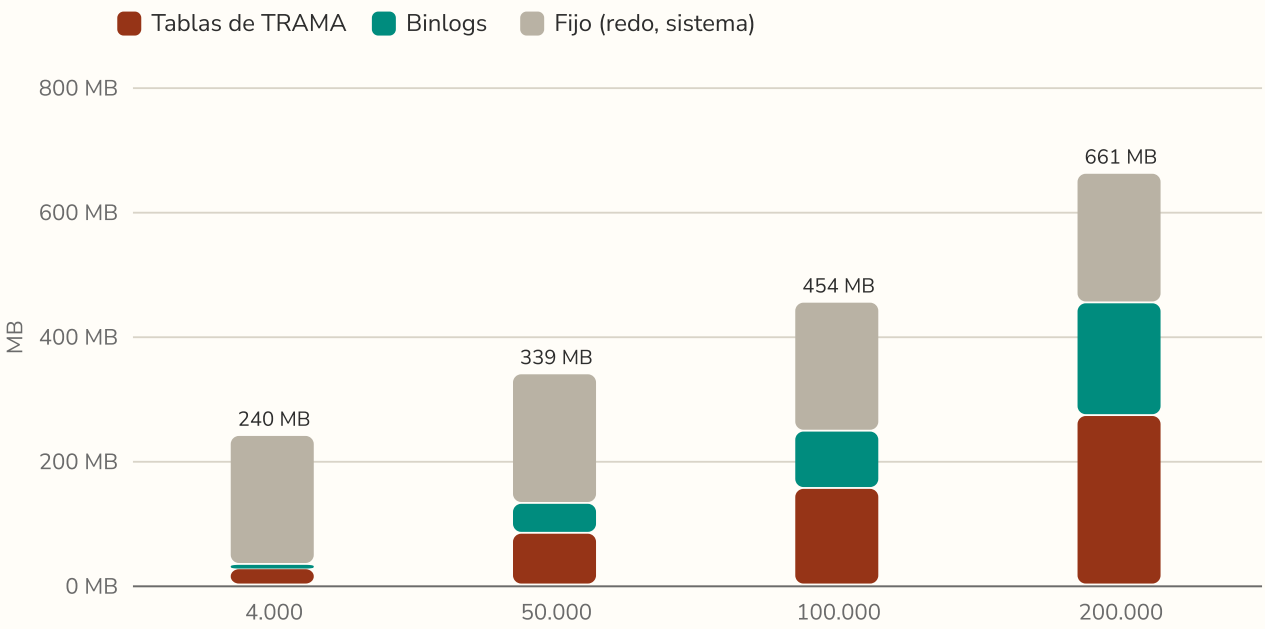


5.1 Cuánto ocupa cada cosa (medido)

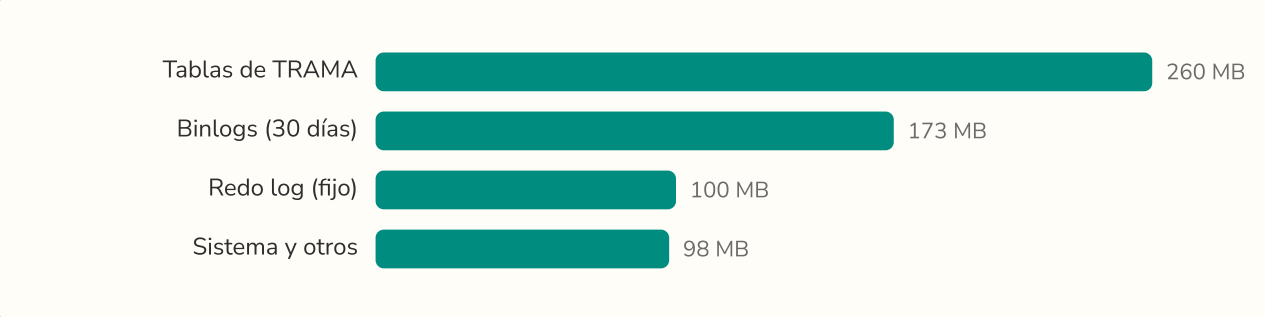
Se cargaron 200.000 aportes, 100 archivos de 1 MB y se hizo un volcado de la base, midiendo el disco en cada paso. MySQL guardó sus datos en un disco virtual con tope de 35 GB, para no llenar la máquina.

Qué	Espacio	Notas
Un aporte	**~1,3 KB** en la base	Incluye índices y su fila en `registro_accion`. Medido entre 50.000 y 200.000 aportes
Un aporte en los binlogs	~0,9 KB	MySQL los guarda **30 días** y después los borra solo
Un archivo adjunto	**su tamaño + 28 bytes**	Se guarda cifrado con AES-GCM; el cifrado casi no agrega
Una vista de perfil	~0,3 KB	Una fila por persona, por niño y por día
Parte fija de MySQL	~210 MB	Redo log (100 MB), tablas del sistema y undo. No crece con los datos
Backup de la base	~0,4 KB por aporte comprimido	200.000 aportes: 130 MB sin comprimir, 79 MB con gzip. El texto cifrado comprime poco
Imágenes de Docker	~2,1 GB	MySQL 1,1 GB, backend 613 MB, OpenBao 259 MB, frontend 82 MB

Aportes	Tablas de TRAMA	Binlogs	MySQL en total
4.000	26 MB	7 MB	240 MB
50.000	83 MB	48 MB	339 MB
100.000	155 MB	92 MB	454 MB
200.000	272 MB	181 MB	661 MB



Cómo crece MySQL con la cantidad de aportes



Dónde ocupa espacio MySQL con 200.000 aportes

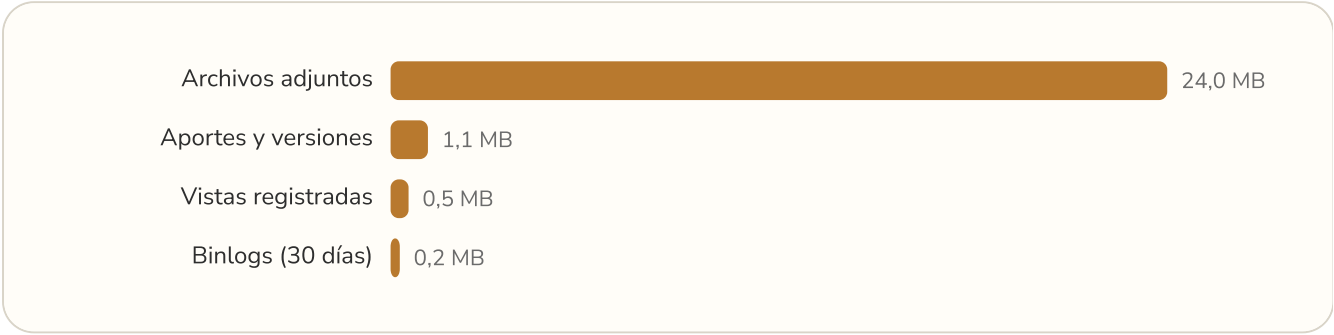
5.2 Supuestos de uso

No hay datos reales de uso todavía. Estos supuestos son a propósito generosos; se pueden cambiar y rehacer la cuenta con la tabla de arriba.

Supuesto (por niño)	Valor
Personas en su red	5
Aportes cargados	2 por día → 730 por año
Aportes editados una vez	20 % → 146 versiones por año
Personas que miran su perfil cada día	las 5 → 1.825 filas de vista por año
Archivos adjuntos	2 por mes de 1 MB → 24 MB por año

Un niño ocupa ~26 MB por año:

Parte	Por año	Porcentaje
Archivos adjuntos	24 MB	**93 %**
Aportes y versiones (876 × 1,3 KB)	1,1 MB	4 %
Vistas (1.825 × 0,3 KB)	0,5 MB	2 %
Binlogs (30 días)	~0,2 MB	1 %



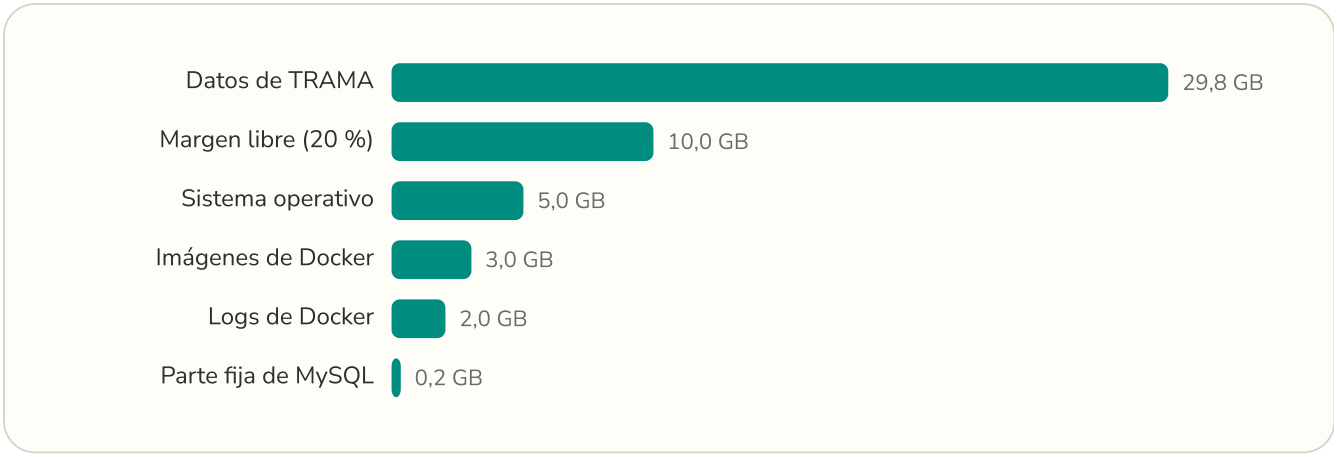
Qué ocupa un niño en un año

Los archivos adjuntos son casi todo el disco. El texto ocupa muy poco.

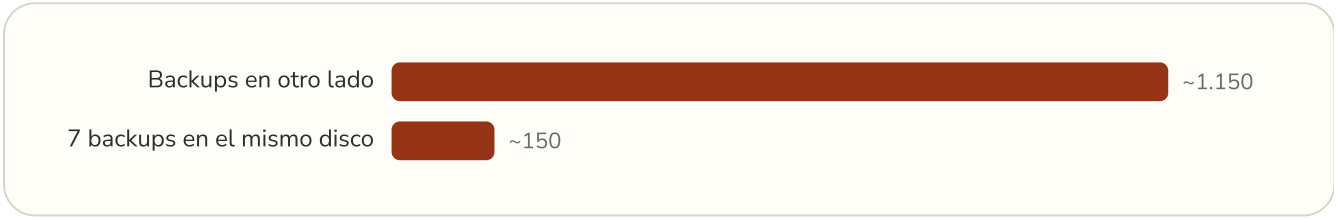
5.3 Cuánto queda libre en un disco de 50 GB

Reserva	Espacio	Por qué
Sistema operativo y paquetes	5 GB	Ubuntu Server con actualizaciones
Imágenes de Docker	3 GB	2,1 GB de imágenes más capas viejas al actualizar
Logs de Docker	2 GB	El compose **no configura rotación de logs**: hoy crecen sin límite
Parte fija de MySQL	0,2 GB	
Margen libre (20 %)	10 GB	**Con el disco lleno TRAMA se cae entera** (capítulo 4 (Casos límite)). No conviene pasar del 80 %
Queda para datos	**~30 GB**	

5.4 Resultado



Cómo se reparte un disco de 50 GB



Niño-años que entran en 50 GB

Escenario	Por niño y año	Capacidad en 30 GB
Backups en otro lado (otro disco, otro servidor o almacenamiento externo)	26 MB	**~1.150 niño-años** (por ejemplo, 575 niños durante 2 años)
7 backups diarios en el mismo disco	26 MB + 7 copias de sus 24 MB de adjuntos ≈ 196 MB	**~150 niño-años**
Sin archivos adjuntos (solo texto)	1,7 MB	~17.500 niño-años
Con el script de backup actual (diario, sin borrar los viejos)	crece sin límite	Con 100 niños con un año de datos (~2,6 GB), cada backup pesa ~2,4 GB: **el disco se llena en unos 12 días**

5.5 Backups

`backup/backup.sh` funciona bien (cifra con AES-256 y no deja copias sin cifrar), pero para la capacidad tiene dos problemas:

1. **No borra backups viejos.** Cada corrida agrega un archivo nuevo en `backups/` y ninguno se elimina.
2. **Cada backup es completo:** la base más **todos** los archivos adjuntos. Además, mientras se arma, copia todos los adjuntos a una carpeta temporal, así que necesita **espacio libre igual al total de adjuntos** solo para correr.

Qué haría:

- Guardar los backups **fuera del servidor** (es lo que de verdad protege ante la pérdida del servidor).
- Si se dejan en el mismo disco, rotar: por ejemplo, 7 diarios y 4 semanales.

- Copiar los adjuntos de forma incremental (por ejemplo con `rsync` o `restic`) en lugar de incluirlos enteros cada vez. Agregar una herramienta nueva requiere acordarlo con el equipo.

5.6 El límite que llega antes que el disco

Con los mismos supuestos, un niño activo llega a **1.000 aportes en unos 16 meses**. Desde ahí su perfil tarda **3,7 s** en 1 CPU · 2 GB (ver capítulo 4 (Casos límite)). **El rendimiento se degrada mucho antes de que el disco se llene**, hasta que se paginen las listas.

5.7 Recomendaciones de operación

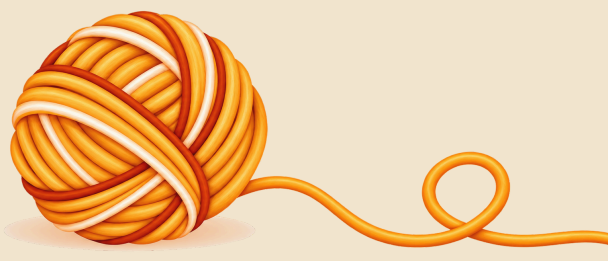
Qué	Cómo
Alerta de disco	Avisar cuando pase el 80 %
Rotación de logs de Docker	`logging: { driver: json-file, options: { max-size: "10m", max-file: "5" } }` en cada servicio
Binlogs	Si no hay réplica, apagarlos (`--skip-log-bin`) o bajar `binlog_expire_logs_seconds` a 3 días
Backups	Fuera del servidor, con rotación
Adjuntos	Si crecen, moverlos a un almacenamiento aparte del disco de la base

TRAMA

CAPÍTULO 6

Seguridad

Código, configuración y dependencias



Revisión del código, la configuración y las dependencias de `develop`. Lo marcado como **verificado** se comprobó con pedidos reales contra el entorno de pruebas; el resto sale de leer el código.

6.1 Lo que está bien

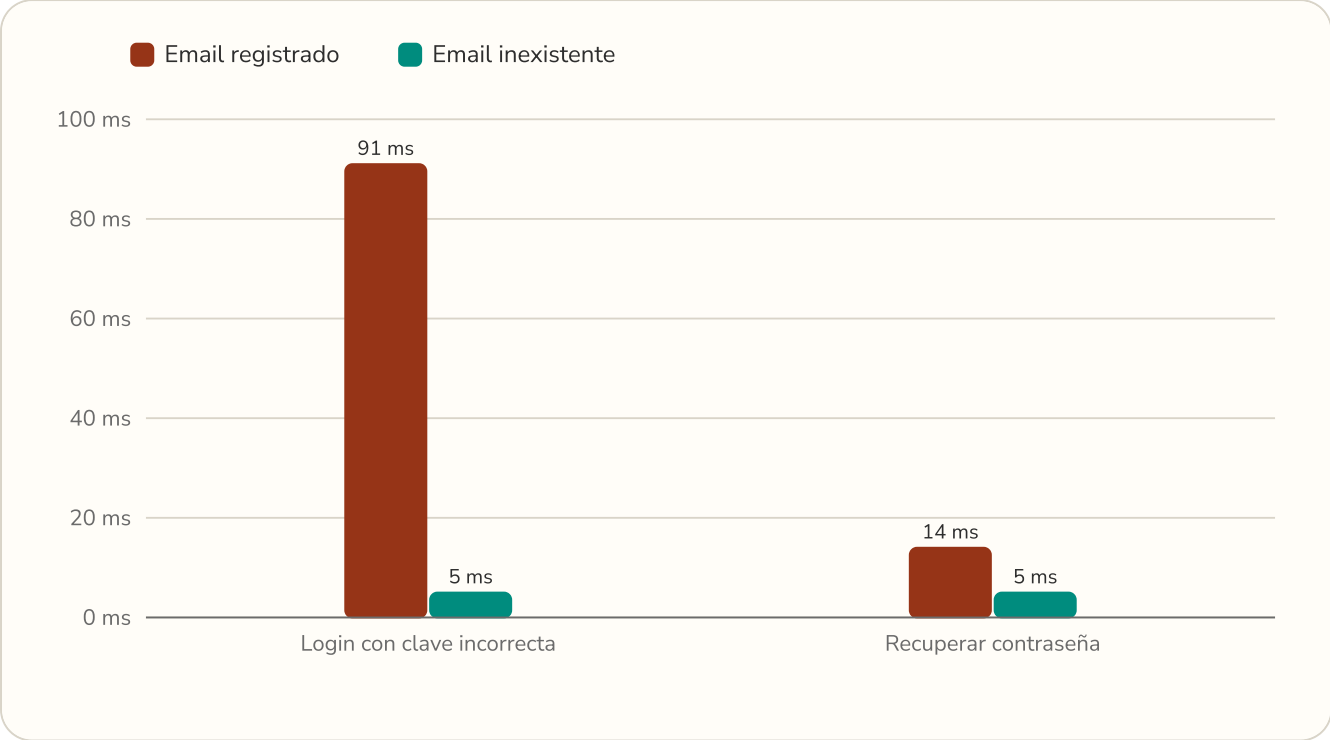
Antes de los problemas, lo que conviene mantener:

- Todo lo que no está declarado en `SecurityConfig` se **deniega** (`anyRequest().denyAll()`), y de Actuator solo se expone `/health`.
- Contraseñas con **BCrypt**, clave del JWT por variable de entorno, sesiones sin estado.
- El filtro JWT revisa que la cuenta siga **habilitada y no bloqueada** en cada pedido.
- **Bloqueo de cuenta** tras 5 intentos fallidos durante 15 minutos.
- `/recuperar` responde **siempre el mismo mensaje**, exista o no el email.
- Archivos adjuntos y datos sensibles **cifrados** (AES); claves de cifrado en OpenBao.
- Las ediciones simultáneas **no corrompen datos**: cada edición exitosa deja su versión (ver capítulo 4 (Casos límite)).

6.2 Código

#	Hallazgo	Gravedad	Evidencia
C1	**Se puede saber si un email está registrado**	Media	**Verificado.** Login con contraseña incorrecta: 91 ms si el email existe y 5 ms si no (no hace BCrypt). Después de 5 intentos el mensaje cambia a `CUENTA_BLOQUEADA` solo si el email existe. `/recuperar`: 14 ms contra 5 ms (con Gmail, segundos de diferencia, porque el mail se envía dentro del pedido)
C2	**Sin límite de pedidos por IP** en `/api/auth/*`	Media-alta	**Verificado.** Con 80 logins por segundo el backend quedó sin responder (capítulo 4 (Casos límite)). Además permite bloquear la cuenta de cualquier persona 15 minutos con 5 intentos (y repetirlo), y disparar mails de registro y recuperación sin freno (agota el cupo de Gmail)
C3	**Errores inesperados sin registrar**	Media	**Verificado.** `GlobalExceptionHandler.handleUnexpected` devuelve `500` sin escribir la excepción en el log. Los 806 `OutOfMemoryError` de las subidas solo se vieron con Java Flight Recorder
C4	**Título del aporte sin cifrar**	Media	`aporte.titulo` y `aporte_version.titulo` no usan `CifradoConverter`. Un título como "Crisis en el recreo" es texto libre sobre el niño; la regla del repo pide cifrarlo
C5	**El JWT sigue valiendo 8 h aunque cambie la contraseña**	Media-baja	No hay lista de revocación ni se compara la fecha del token con la del último cambio de contraseña. Si alguien roba una sesión, restablecer la contraseña no la corta
C6	**Contraseñas de más de 72 bytes**	Baja	Spring Security 6.1.9 tiene CVE-2025-22228: BCrypt solo compara los primeros 72 bytes. `RegisterRequest` no pone un largo máximo a la contraseña
C7	**Código de recuperación adivinable con paciencia**	Baja	6 dígitos, 5 intentos por código y un código nuevo por minuto: ~7.200 intentos por día y cuenta (~0,7 % de acertar por día). Se combina con C2

#	Hallazgo	Gravedad	Evidencia
C8	**Tipo de archivo según lo que declara el navegador**	Baja	`ArchivoService` valida extensión y `Content-Type` declarado, no el contenido real del archivo. La foto de perfil se sirve sin `X-Content-Type-Options: nosniff` (el comprobante sí lo tiene)
C9	**Mails dentro del pedido**	Baja (rendimiento)	No hay `@Async`: registro, invitaciones, recuperación y avisos esperan a Gmail. Un Gmail lento frena esos pedidos y retiene su conexión a la base



Tiempo de respuesta según si el email existe

Qué haría:

- **C1:** responder igual con email inexistente, ejecutando BCrypt contra un hash fijo; mismo mensaje para cuenta bloqueada e inexistente; mails en segundo plano.
- **C2:** límite por IP y por email en `/api/auth/*`. Se puede hacer en nginx (`limit_req`) sin dependencias nuevas.
- **C3:** registrar la excepción en `handleUnexpected` (sin datos del niño).
- **C4:** cifrar el título. Es un cambio de esquema: migración Flyway y migrar los títulos existentes. Para decidir con el equipo.
- **C5:** guardar la fecha del último cambio de contraseña y rechazar tokens emitidos antes.
- **C6:** `@Size(max = 72)` en la contraseña y actualizar Spring Security.
- **C8:** validar los primeros bytes del archivo (PNG, JPEG, PDF) y agregar `nosniff` a la foto.

6.3 Configuración y despliegue

#	Hallazgo	Gravedad	Evidencia
D1	**MySQL y el backend publicados en todas las interfaces**	Alta (en un VPS)	`docker-compose.yml`: `"3307:3306"` y `"4002:8080"`. En un servidor con IP pública quedan abiertos a internet: Docker escribe sus propias reglas de firewall y se saltea `ufw`
D2	**Demo pública con cuentas de contraseña conocida**	Alta (en la demo)	`backend-demo` corre con `SPRING_PROFILES_ACTIVE: dev`, que activa `DataSeeder`: crea `tutor@trama.test`, `docente@trama.test` y otras con la contraseña fija `Trama1234!`. La demo está publicada por ngrok
D3	**Logs de Docker sin rotación**	Media (operación)	Ningún servicio configura `logging`: los logs crecen hasta llenar el disco
D4	**Contenedores corriendo como root**	Baja	Ningún `Dockerfile` usa `USER`
D5	**nginx sin cabeceras de seguridad**	Baja	`nginx.conf.template` no agrega `Content-Security-Policy`, `X-Frame-Options`, `Referrer-Policy` ni oculta la versión (`server_tokens off`)
D6	**Backups sin rotación**	Media (operación)	Ver capítulo 5 (Capacidad en 50 GB)

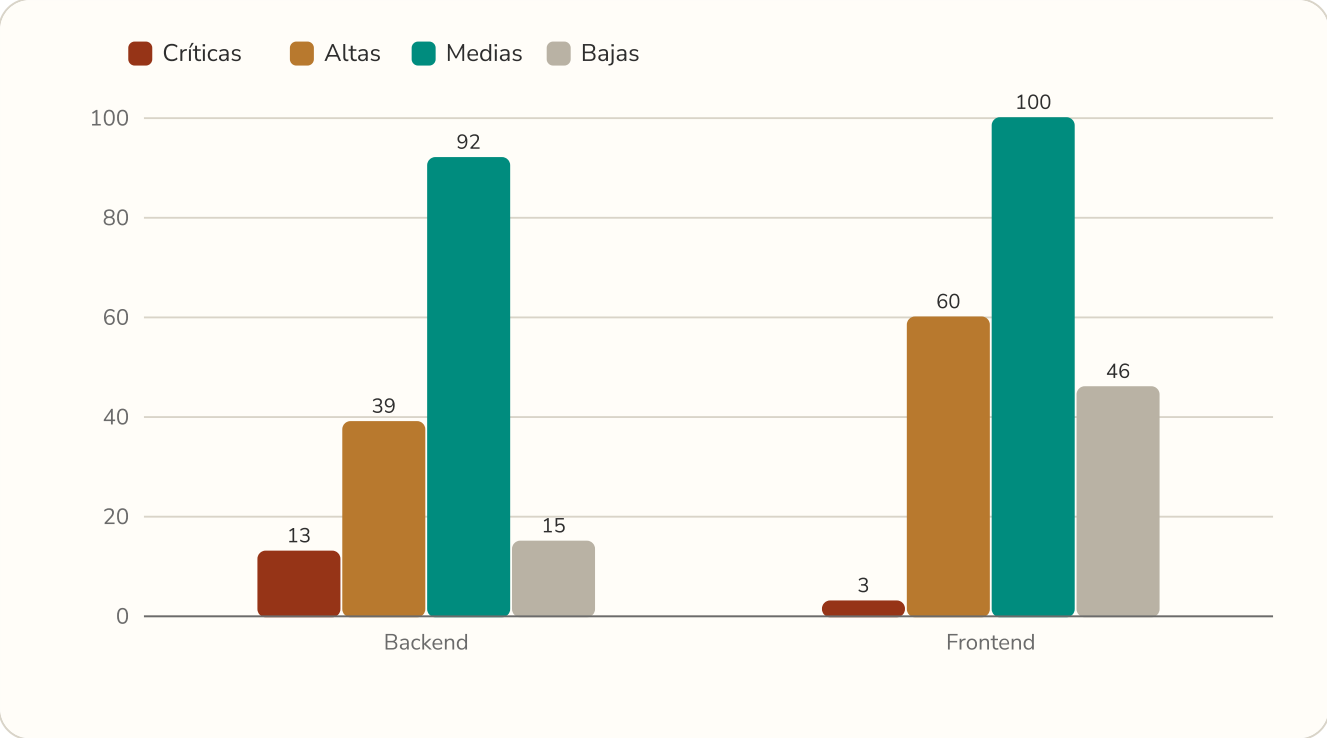
Qué haría:

- **D1:** `"127.0.0.1:3307:3306"` y `"127.0.0.1:4002:8080"`, como ya hacen Mailpit, OpenBao y la demo.
- **D2:** un perfil `demo` sin `DataSeeder`, o que el seeder solo corra con una variable explícita.
- **D3:** `logging.options.max-size` y `max-file` en cada servicio.
- **D4:** `USER` sin privilegios en los `Dockerfile`.
- **D5:** agregar las cabeceras en nginx.

6.4 Dependencias (Trivy)

Escaneo de las imágenes con Trivy 0.57.1, que descargó su base de vulnerabilidades al momento del escaneo (6 de noviembre de 2026):

Imagen	Críticas	Altas	Medias	Bajas
`trama-backend` (Ubuntu 26.04 + Java)	**13**	39	92	15
`trama-frontend` (nginx 1.27 sobre Alpine 3.21.3)	3	60	100	46



Vulnerabilidades conocidas por imagen

Backend

La causa de fondo es **Spring Boot 3.1.11, que ya no tiene soporte gratuito**. Arrastra versiones viejas de Tomcat (10.1.20), Spring Framework (6.0.19), Spring Security (6.1.9) y Jackson (2.15.4).

Componente	CVE	¿Aplica a TRAMA?
Spring Security 6.1.9	CVE-2025-22228 (BCrypt y 72 bytes)	**Sí**, ver C6
Tomcat 10.1.20	CVE-2025-48988 (DoS en subidas *multipart*)	**Sí**: TRAMA acepta subidas
Tomcat 10.1.20	CVE-2024-38286, CVE-2025-52520, CVE-2025-53506, CVE-2026-41284 (denegación de servicio)	**Probablemente sí**
Tomcat 10.1.20	CVE-2025-24813 (escritura parcial con PUT)	No: el servlet por defecto de Spring Boot es de solo lectura
Tomcat 10.1.20	CVE-2026-41293, CVE-2025-48989 y otras de HTTP/2	No: Spring Boot no habilita HTTP/2 por defecto
Tomcat 10.1.20	CVE-2026-43512, CVE-2026-68525 y otras de autenticación FORM o *digest*	No: TRAMA usa JWT
Tomcat 10.1.20	CVE-2024-50379 / CVE-2024-56337 (JSP en sistemas de archivos sin distinción de mayúsculas)	No: no hay JSP y corre en Linux
Tomcat 10.1.20	CVE-2026-65182 (evasión de restricciones de seguridad)	**A revisar**
Spring Framework 6.0.19	CVE-2026-47884 (ejecución remota en `spring-webmvc`)	**A revisar.** Trivy la marca crítica para esta versión; no pude verificar las condiciones exactas

Componente	CVE	¿Aplica a TRAMA?
Spring Security 6.1.9	CVE-2026-22732 (evasión de políticas)	**A revisar**
Spring Security 6.1.9	CVE-2024-38821	No: solo afecta a WebFlux
Spring Framework 6.0.19	CVE-2024-38816 / CVE-2024-38819 (*path traversal*)	No: solo con `RouterFunctions` y recursos estáticos
Spring Boot 3.1.11	CVE-2025-22235 (`EndpointRequest.to()`)	No: no se usa
Jackson 2.15.4	Varias de deserialización polimórfica	Probablemente no: TRAMA no activa *default typing*. Las de denegación de servicio sí podrían aplicar
BouncyCastle 1.73	CVE-2025-14813, CVE-2026-8763, CVE-2026-5598	**A revisar** qué dependencia la trae y si se usa
snakeyaml 1.33	CVE-2022-1471	No: solo se leen los YAML propios
mysql-connector-j 8.0.33	CVE-2023-22102	No en la práctica: requiere un servidor MySQL malicioso

Qué haría: actualizar Spring Boot a una versión con soporte. Resuelve casi todas de una vez. Como son cambios de versión mayores, conviene una rama propia con todos los tests (`./mvnw test`) y repetir estas pruebas de carga.

Frontend

Las 3 críticas y la mayoría de las altas son paquetes de Alpine 3.21.3 dentro de `nginx:1.27-alpine`: OpenSSL (la crítica solo afecta a sistemas de 32 bits), brotli, curl y c-ares. **Se resuelven reconstruyendo con una imagen base de nginx actualizada**, sin tocar código.

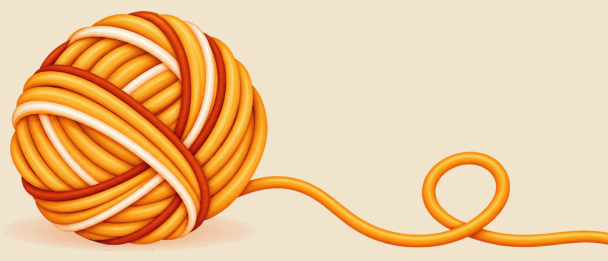
`npm audit` sobre las dependencias de producción marca además `follow-redirects` (alta), que viene con axios. En el navegador axios no lo usa, así que el riesgo real es bajo, pero se arregla con `npm audit fix`.

TRAMA

CAPÍTULO 7

Escalabilidad

Qué escala, qué no, por qué y cómo podría escalar



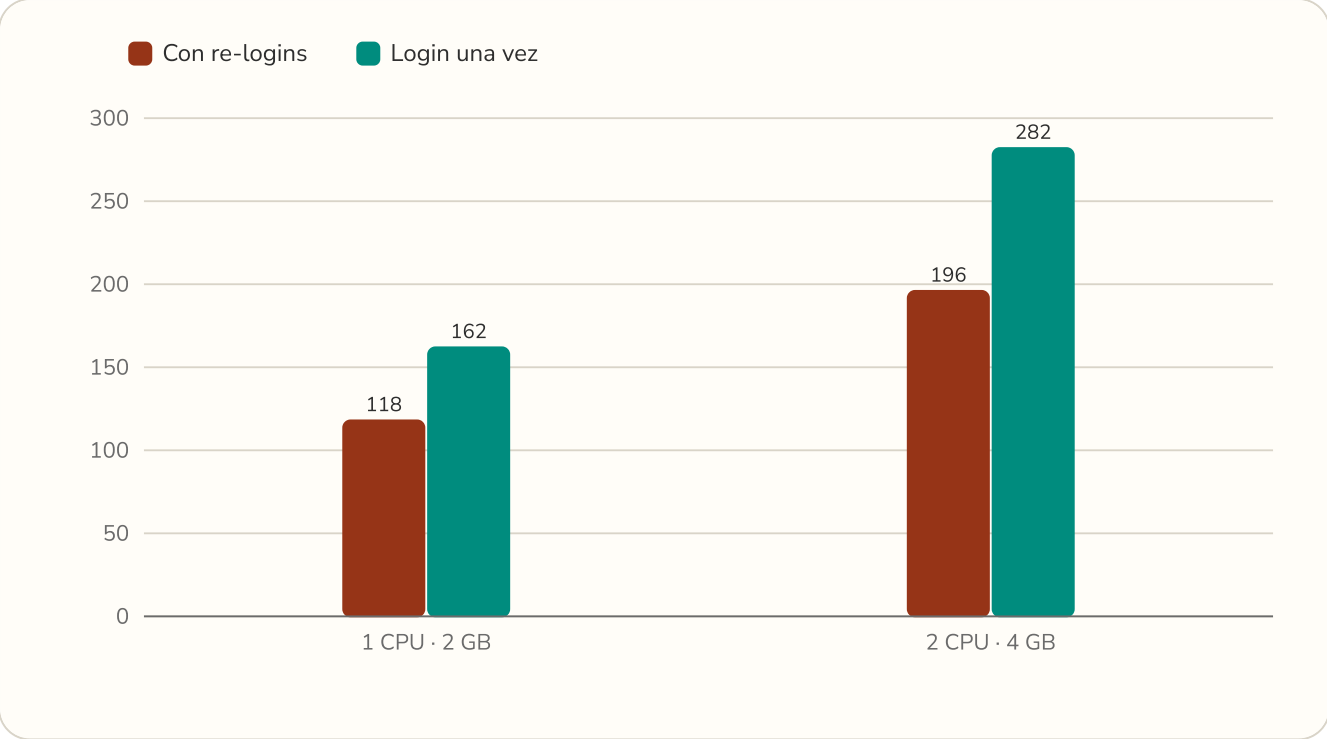
7.1 Respuesta corta

TRAMA escala bien en cantidad de familias y de datos totales, escala a medias agrandando el servidor, no escala en aportes por niño y hoy no puede tener más de una copia del backend.

Lo que está medido y lo que es estimación se marca en cada caso. Las estimaciones salen de las mediciones de los capítulos anteriores, no de supuestos sueltos.

Dimensión	¿Escala?	Evidencia
Datos totales en la base	Sí	Con 200.000 aportes MySQL respondió igual de rápido; ninguna consulta pasó de 0,2 ms. **Medido**
Cantidad de familias y niños	Sí	Cada niño es independiente; sumar niños solo suma datos, que MySQL maneja bien. **Medido** con 200 niños
Agrandar el servidor (vertical)	A medias	El doble de CPU dio entre un 66 % y un 74 % más de pedidos por segundo, no el doble. **Medido**
Aportes por niño	No	Cada pedido trae todos los aportes del niño: con 1.000, abrir un perfil tarda 3,7 s; con 5.000, 16 s. **Medido**
Más copias del backend (horizontal)	Hoy no	Archivos en el disco local y tareas programadas que se repetirían en cada copia. **Leído en el código**
Logins a la vez	No	~16 a 20 logins por segundo como máximo y, pasado eso, todo el sistema espera. **Medido**
Mails	A medias	Se envían dentro del pedido: con Gmail lento, el pedido espera y retiene su conexión. **Leído en el código**
Disco	A medias	Crece con los adjuntos y los backups completos sin rotación; lleno, la app cae entera. **Medido**

7.2 Escalado vertical: agrandar el servidor



Escalado vertical medido

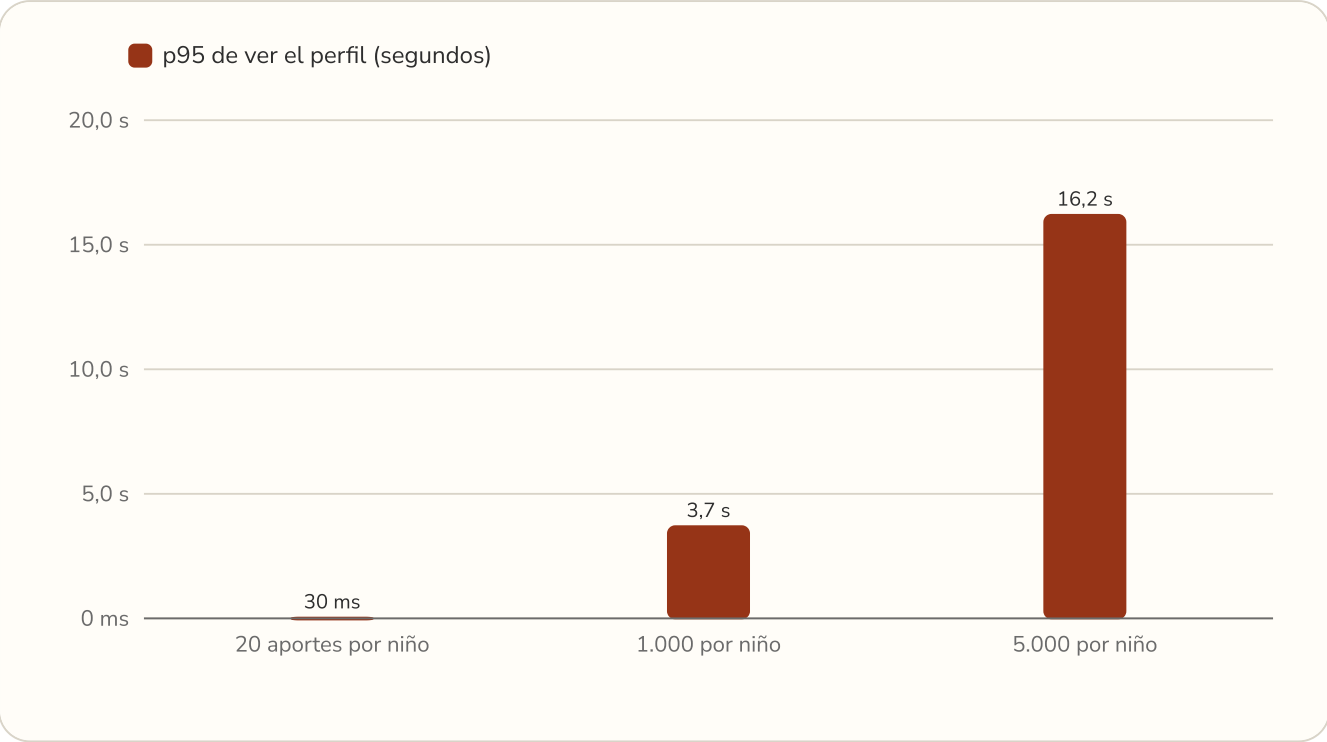
Pasar de 1 CPU · 2 GB a 2 CPU · 4 GB	Antes	Después	Mejora
Techo con re-logins	118 pedidos/s	196 pedidos/s	+66 %
Techo con login una vez	162 pedidos/s	282 pedidos/s	+74 %
p95 con 200 usuarios	1,4 s	234 ms	6 veces mejor

Por qué no rinde el doble:

1. **Las 10 conexiones a la base no crecen con el servidor.** Con más CPU se atienden más pedidos a la vez, pero siguen turnándose las mismas 10 conexiones: con 600 usuarios en 4 GB los pedidos sumaron 11.150 s de espera por conexión en un minuto.
2. **MySQL y el backend comparten la CPU.** En 4 GB, MySQL usó hasta el 40 % de un núcleo.

Hasta dónde conviene agrandar (estimación): con el código de hoy, pasar a 4 CPU · 8 GB daría otro +50 % a +70 % de techo, siempre que se suban también las conexiones (`spring.datasource.hikari.maximum-pool-size`). Pasado eso, el costo crece más rápido que el beneficio, y **ningún servidor resuelve el problema de los aportes por niño**.

7.3 El problema que no se resuelve con más servidor: aportes por niño



Cuánto empeora al crecer cada dimensión

Es la dimensión que **no escala** y la más importante, porque crece sola con el uso: cada niño suma aportes todos los días.

Por qué no escala. El costo de abrir un perfil es **proporcional a la cantidad de aportes del niño**, porque:

Paso	Costo con 20 aportes	Costo con 5.000 aportes
Traer los aportes de la base	20 filas	5.000 filas
Buscar los adjuntos de cada uno (N+1)	20 consultas	**5.000 consultas**
Descifrar contenido y datos (AES)	40 descifrados	10.000 descifrados
Armar el JSON y mandarlo	~20 KB	~5 MB

Un niño activo llega a 1.000 aportes en unos 16 meses con los supuestos de capítulo 5 (Capacidad en 50 GB). **En un servidor de cualquier tamaño, el tiempo sigue creciendo con cada aporte nuevo.**

Cómo podría escalar. Haciendo que el costo de cada pedido **no dependa** de cuántos aportes tiene el niño:

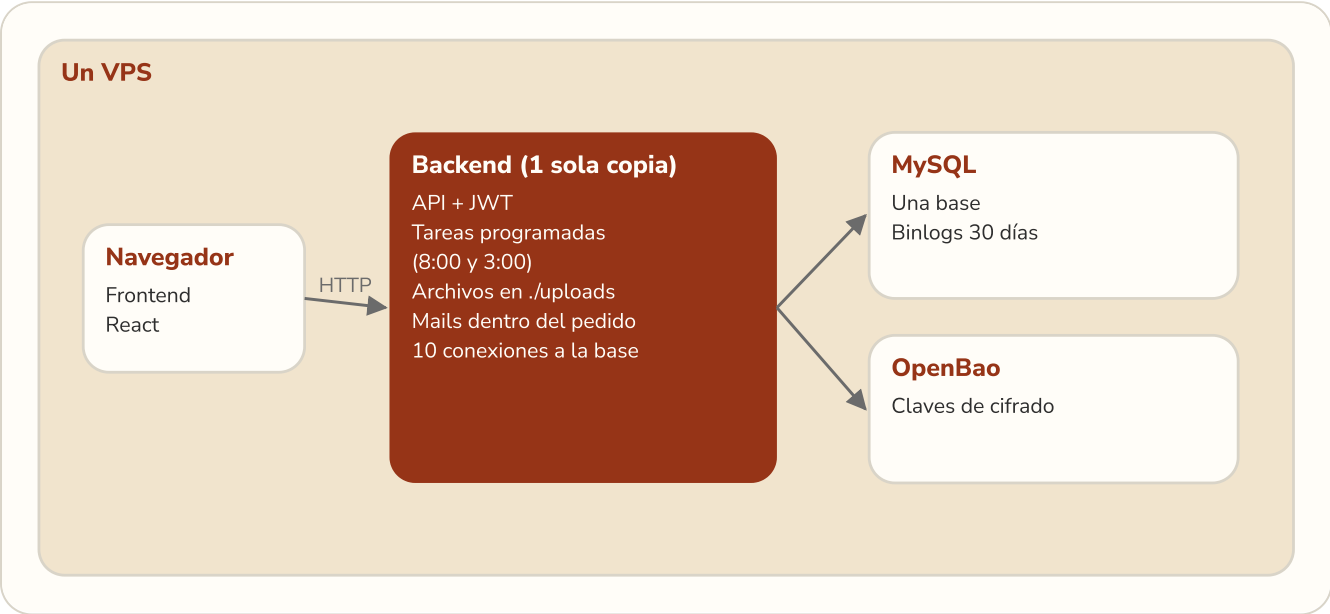
1. **Paginar** `GET /aportes` y `GET /bitacora`: la pantalla pide de a 50, ordenados por fecha, y carga más al bajar. Spring Data lo resuelve con `Pageable` y una respuesta `Page`. Requiere el cambio correspondiente en el frontend.
2. **Sacar el N+1 de adjuntos.** La forma más rápida, sin tocar código, es una propiedad de Hibernate que trae los adjuntos de varios aportes juntos: ``` `properties spring.jpa.properties.hibernate.default_batch_fetch_size=50` ``` Con eso, 5.000 consultas pasan a

`ser unas 100. La forma prolija es traerlos en la misma consulta (`@EntityGraph`o`JOIN FETCH`)` solo para la página pedida.

3. **Índice para la página:** `(proyecto_id, archivado, creado_en)` en `aporte`, para que pedir "los 50 más recientes" no recorra todos. Requiere una migración nueva.

Con las tres cosas, abrir un perfil cuesta lo mismo con 20 que con 20.000 aportes (estimación: el mismo orden que hoy con 20 aportes, unas decenas de milisegundos).

7.4 Escalado horizontal: varias copias del backend



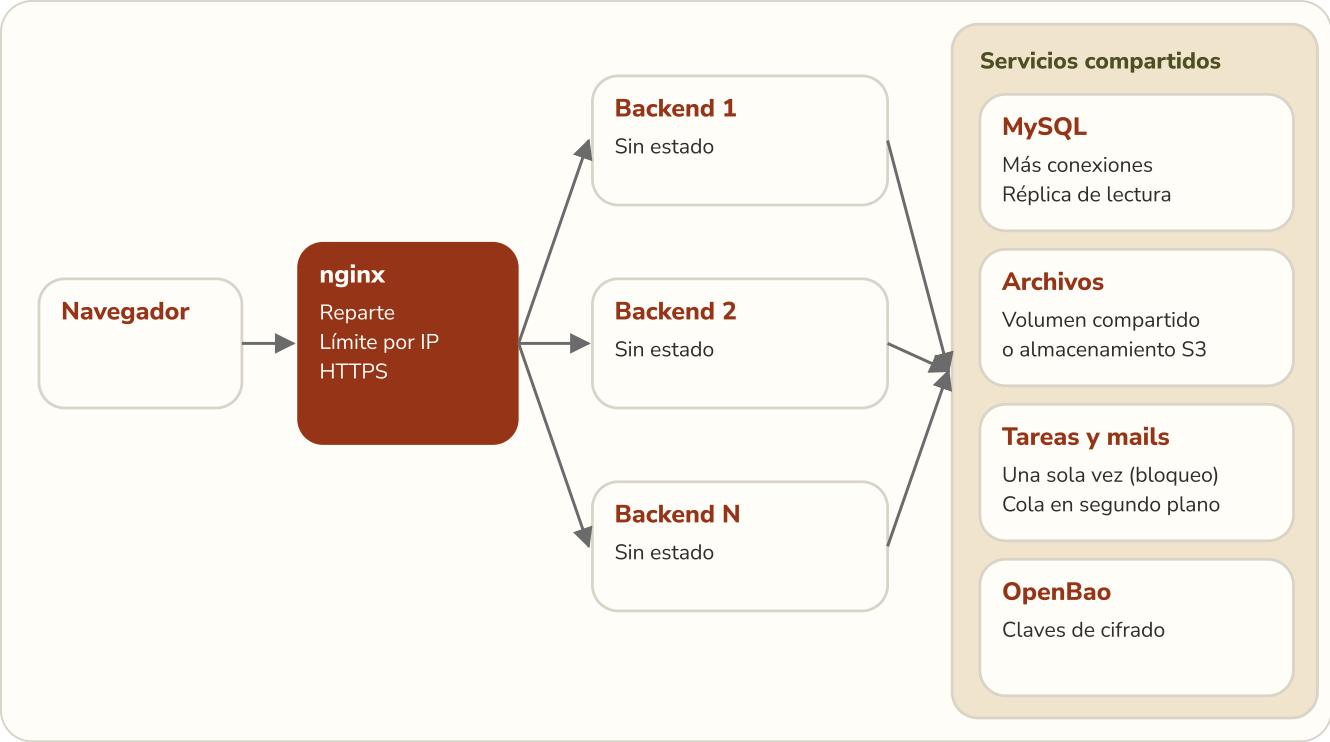
Arquitectura de hoy

La buena noticia es que **TRAMA ya es casi "sin estado"**: el JWT se valida con una clave compartida, no hay sesiones HTTP ni datos guardados en memoria, y los bloqueos de cuentas y códigos viven en la base. Eso es lo más difícil de lograr y ya está.

Lo que hoy impide tener dos o más copias:

Bloqueo	Por qué	Qué pasaría con 2 copias
Archivos en el disco local	`ArchivoService` guarda en `trama.archivos.directorio` (`./uploads`) del propio contenedor	Una foto subida a la copia 1 no existe para la copia 2: la mitad de las descargas darían "archivo no encontrado"
Tareas programadas	`AvisoAcuerdoService` (todos los días a las 8:00) y `EliminacionProyectoService` (a las 3:00) usan `@Scheduled`	Cada copia las ejecuta: **mails de acuerdos vencidos duplicados** y dos procesos borrando los mismos proyectos a la vez
Mails dentro del pedido	No hay `@Async` ni cola	No impide escalar, pero cada copia retiene conexiones mientras espera a Gmail
Sin reparto de pedidos	El compose publica el backend directo, sin un nginx adelante	No hay quién reparta entre copias

Cómo podría escalar horizontalmente:



Cómo podría escalar

1. **nginx adelante** (ya está en el repo para el frontend), repartiendo `/api` entre copias con `upstream`. De paso, resuelve el límite por IP y HTTPS (ver capítulo 8 (Cómo arreglar las vulnerabilidades)).
2. **Archivos fuera del contenedor.** Opción simple: un volumen compartido montado en todas las copias (en un solo servidor alcanza). Opción a futuro: un almacenamiento de objetos compatible con S3. Los archivos ya se guardan cifrados, así que el proveedor no ve el contenido.
3. **Tareas programadas una sola vez.** Opciones, de menor a mayor esfuerzo: activarlas solo en una copia con una variable (`TRAMA_TAREAS_ACTIVAS=true` y `@ConditionalOnProperty`); o un bloqueo en la base para que solo una copia las ejecute. Existe una librería para esto (ShedLock), pero agregarla requiere acordarlo con el equipo.
4. **Mails en segundo plano**, con `@Async` o una tabla de mails pendientes que procese una tarea. Así el pedido responde sin esperar a Gmail.
5. **Conexiones a MySQL:** cada copia abre 10. MySQL acepta 151 por defecto, así que entran unas 12 copias antes de tocar su configuración.

7.5 Logins: el cuello de botella de BCrypt

BCrypt es lento **a propósito**. No hay que hacerlo más rápido: hay que evitar que trabe al resto.

Problema	Por qué	Cómo podría escalar
El login retiene una conexión mientras calcula BCrypt	Open Session In View activado	`spring.jpa.open-in-view=false`: la conexión se suelta apenas se lee el usuario
Un pico de logins frena a todos	Comparten la misma CPU y las mismas 10 conexiones	Límite por IP en nginx; separar logins a su propio grupo de hilos si hiciera falta
Cada login consume ~100 ms de CPU	Es el costo de BCrypt con fuerza 10	Mantenerlo: es la protección. Con varias copias, el costo se reparte

Sacando los re-logins, el techo **medido** subió un 37 % en 2 GB y un 44 % en 4 GB. Con `open-in-view=false` se espera que la saturación sea **gradual** en vez de brusca, porque el resto de los pedidos ya no esperaría una conexión retenida por BCrypt (estimación; conviene medirlo después del cambio).

7.6 Qué ganaría cada cambio

Cambios ordenados por cuánto mejoran la escalabilidad respecto del esfuerzo. Las mejoras **medidas** vienen de las pruebas; las **estimadas** se calcularon con las grabaciones de Java Flight Recorder y deberían confirmarse midiendo después del cambio.

Cambio	Esfuerzo	Qué mejora	Tipo
Paginación + adjuntos en una consulta	Medio (backend + frontend)	El perfil deja de empeorar con cada aporte	Estimado, sobre medición
`default_batch_fetch_size=50`	Una línea	Las listas pasan de N+1 a ~N/50 consultas	Estimado
`open-in-view=false`	Una línea + revisar tests	La saturación deja de ser brusca	Estimado
Menos re-logins (el token dura 8 h)	Ninguno en el backend	+37 % a +44 % de techo	**Medido**
Pasar de 1 CPU · 2 GB a 2 CPU · 4 GB	Costo mensual	+66 % a +74 % de techo	**Medido**
Permisos una vez por pedido (hoy ~4,5)	Bajo	~15 % menos consultas por pedido	Estimado
Filtro JWT: un solo parseo del token	Bajo	~8 % menos de CPU en pedidos livianos (jjwt usa hoy el 12 %)	Estimado
Archivos fuera del disco + tareas una vez + nginx	Medio	Habilita varias copias del backend	Requisito

7.7 Conclusión

Para el tamaño de un lanzamiento (decenas o pocos cientos de familias activas), **un servidor de 1 CPU · 2 GB alcanza con holgura**, y el de 2 CPU · 4 GB da margen para crecer. El riesgo real no es la cantidad de gente sino **el tiempo**: a medida que cada niño acumula aportes, la app se vuelve lenta para esa familia, sin importar el servidor.

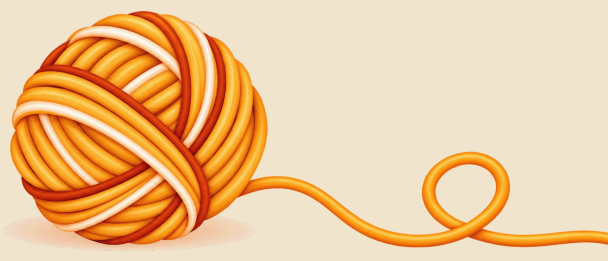
Por eso, antes de pensar en más servidores, lo que más escalabilidad da es **paginar las listas**. Después, cuando haga falta crecer de verdad, los cambios para tener varias copias del backend son pocos y concretos, porque la base sin estado ya está.

TRAMA

CAPÍTULO 8

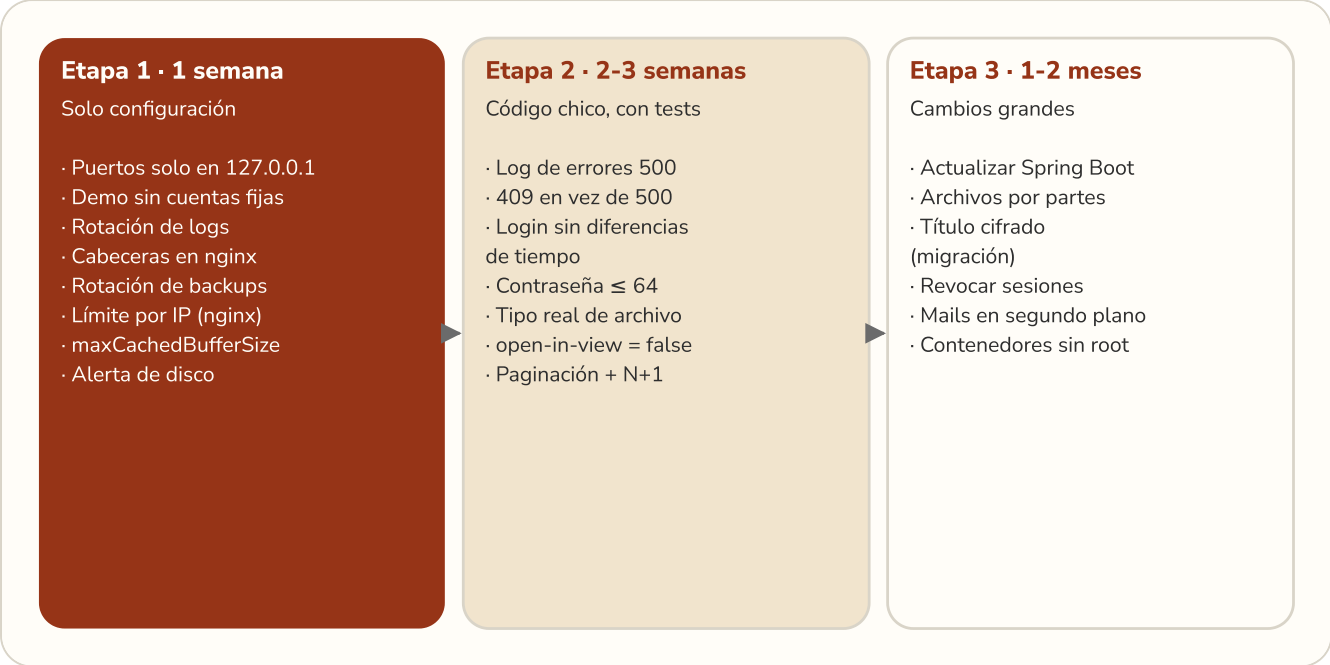
Cómo arreglar las vulnerabilidades

Tres etapas, con código de ejemplo y cómo verificar cada arreglo



Esto es una propuesta. Ningún cambio de este capítulo está aplicado. Los fragmentos de código muestran el arreglo sobre los archivos reales de TRAMA, pero cada uno tiene que pasar por su tarjeta, sus tests (caso bueno, caso de error y `403` donde corresponda) y su PR, como pide `AGENTS.md`.

8.1 Plan en tres etapas



Hoja de ruta



Esfuerzo contra impacto

Etapa	Qué incluye	Toca código Java	Riesgo de romper algo
1 · Configuración (1 semana)	D1, D2, D3, D5, D6, C2, archivos (paliativo)	No	Bajo
2 · Código chico (2-3 semanas)	C1, C3, C6, C8, *deadlocks*, `open-in-view`	Sí, acotado	Medio: cubrir con tests
3 · Cambios grandes (1-2 meses)	Dependencias, archivos por partes, C4, C5, C9, D4	Sí	Alto: rama propia y repetir estas pruebas

8.2 Etapa 1: configuración

D1 · Puertos abiertos a internet

Arreglo en `docker-compose.yml`, igual que ya están Mailpit, OpenBao y la demo:

```
mysql:
  ports:
    - "127.0.0.1:3307:3306"
backend:
  ports:
    - "127.0.0.1:4002:8080"
```

Verificar: desde otra máquina de la red, `nc -vz <ip-del-servidor> 3307` tiene que fallar.

D2 · Demo pública con cuentas de contraseña fija

Arreglo: que `DataSeeder` no corra en la demo. Lo más simple, sin tocar el perfil `dev` que usa el equipo, es pedir una variable explícita:

```
@Component
@Profile("dev")
@ConditionalOnProperty(name = "trama.datos-de-ejemplo", havingValue = "true", matchIfMissing =
true)
public class DataSeeder implements CommandLineRunner {
```

Y en `backend-demo` del compose: `TRAMA_DATOS_DE_EJEMPLO: "false"`.

Verificar: con la demo levantada, el login con `tutor@trama.test` / `Trama1234!` tiene que dar `401`. Si la base de la demo ya tiene esas cuentas, hay que borrarlas o cambiarles la contraseña.

D3 · Logs sin rotación

Arreglo en `docker-compose.yml`, una vez arriba y referenciado en cada servicio:

```
x-logs: &logs
  driver: json-file
  options: { max-size: "10m", max-file: "5" }

services:
  backend:
    logging: *logs
```

D5 · Cabeceras de seguridad en nginx

Arreglo en `frontend/nginx.conf.template`, dentro de `server`:

```
server_tokens off;
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "DENY" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Content-Security-Policy "default-src 'self'; img-src 'self' data: blob:; style-src
'self' 'unsafe-inline' https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com"
always;
```

La política de contenido (CSP) hay que probarla con el frontend real: si algo deja de cargar, la consola del navegador dice qué falta permitir.

D6 · Backups sin rotación

Arreglo al final de `backup/backup.sh`:

```
# Guardar los últimos 7 backups y borrar los más viejos
ls -1t /backups/trama-*.tar.gz.gpg 2>/dev/null | tail -n +8 | xargs -r rm -f
```

Lo ideal es además **copiar cada backup fuera del servidor**: un backup en el mismo disco no protege si se pierde el servidor.

C2 · Sin límite de pedidos por IP

Arreglo en nginx, sin dependencias nuevas. Va al principio de `nginx.conf.template` (fuera del bloque `server`, porque se incluye dentro de `http`):

```
limit_req_zone $binary_remote_addr zone=auth:10m rate=10r/m;
```

Y en el `server`, una ubicación propia para la autenticación:

```
location /api/auth/ {
    limit_req zone=auth burst=5 nodelay;
    limit_req_status 429;
    set $backend_trama ${DEMO_BACKEND_URL};
    proxy_pass $backend_trama;
    # mismas cabeceras proxy_set_header que en /api/
}
```

Hoy en producción el backend se publica directo. Para que esto proteja, **el tráfico tiene que pasar por nginx** (ver capítulo 7 (Escalabilidad)).

Verificar: repetir la tormenta de logins (`./limites.sh 1cpu-2gb logins`). Desde una sola IP, después de unos pocos intentos por minuto tienen que llegar `429` y el backend tiene que seguir respondiendo.

Archivos que fallan después de un pico (paliativo)

Mientras no se cambie `ArchivoService` (etapa 3), una opción de Java evita que cada hilo guarde un buffer del tamaño del archivo:

```
  backend:
    environment:
      JAVA_TOOL_OPTIONS: "-Djdk.nio.maxCachedBufferSize=262144"
```

Verificar: repetir `./limites.sh 2cpu-4gb edicion textos archivos`. Las subidas tienen que dar `201` aun después de la edición simultánea.

8.3 Etapa 2: código chico

C3 · Errores 500 sin registrar

Arreglo en `GlobalExceptionHandler.handleUnexpected`. Se registra el tipo y la pila, **nunca** el cuerpo del pedido (puede tener datos del niño):

```
private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);

@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleUnexpected(Exception e, HttpServletRequest request) {
    log.error("Error inesperado en {} {}", request.getMethod(), request.getRequestURI(), e);
    // ... la misma respuesta 500 de hoy
}
```

Test: un controlador de prueba que lance una excepción y verificar que la respuesta sigue siendo `500 ERROR_INTERNO`.

Ediciones simultáneas: `500` por *deadlock*

Arreglo en `GlobalExceptionHandler`, una respuesta clara en vez de `500`:

```
@ExceptionHandler({ CannotAcquireLockException.class,
ObjectOptimisticLockingFailureException.class })
public ResponseEntity<ErrorResponse> handleConflicto(Exception e) {
    return ResponseEntity.status(HttpStatus.CONFLICT).body(new ErrorResponse(
            "EDICION_EN_CONFLICTO",
            "Este aporte se acaba de guardar desde otro lado. Revisá los cambios y volvé a
intentar."));
}
```

Verificar: repetir `./limites.sh 2cpu-4gb edicion`: los fallidos tienen que ser `409`, y las versiones en la base tienen que seguir coincidiendo con las ediciones guardadas.

C1 · Se puede saber si un email está registrado

Arreglo en `AuthenticationService.authenticate`: cuando el email no existe, hacer igual un BCrypt contra un hash fijo, para que tarde lo mismo:

```
// Hash de una contraseña al azar, calculado una vez en el constructor
// (no en la declaración del campo: ahí passwordEncoder todavía es null)
private final String hashFicticio;
// en el constructor: this.hashFicticio = passwordEncoder.encode(UUID.randomUUID().toString());

Usuario usuario = repository.findByEmail(email).orElse(null);
if (usuario == null) {
    passwordEncoder.matches(request.getPassword(), hashFicticio);   // mismo costo que un email
real
    throw new BadCredentialsException("Credenciales inválidas");
}
```

El mensaje `CUENTA_BLOQUEADA` también delata la cuenta. Con el límite por IP (C2) ese dato se vuelve difícil de explotar; si se quiere cerrar del todo, conviene responder lo mismo que ante credenciales inválidas y avisarle a la persona por mail que su cuenta se bloqueó.

Verificar: repetir la medición de 20 emails registrados y 20 inexistentes: las medianas tienen que quedar parecidas.

C6 · Contraseñas de más de 72 bytes

Arreglo en `RegisterRequest` y `RestablecerContrasenaRequest`. Se pone 64 caracteres y no 72, porque un carácter con tilde ocupa más de un byte:

```
@Size(min = 8, max = 64, message = "La contraseña debe tener entre 8 y 64 caracteres")
private String password;
```

Requiere el mismo límite en el formulario del frontend.

C8 · Tipo real del archivo y `nosniff`

Arreglo en `ArchivoService`: además de la extensión, revisar los primeros bytes:

```
private static boolean coincideFirma(byte[] b, String mime) {
    return switch (mime) {
        case "image/png" -> b.length > 4 && (b[0] & 0xFF) == 0x89 && b[1] == 'P' && b[2] == 'N' &&
b[3] == 'G';
        case "image/jpeg" -> b.length > 3 && (b[0] & 0xFF) == 0xFF && (b[1] & 0xFF) == 0xD8 &&
(b[2] & 0xFF) == 0xFF;
        case "application/pdf" -> b.length > 4 && b[0] == '%' && b[1] == 'P' && b[2] == 'D' &&
b[3] == 'F';
        default -> false;
    };
}
```

Y en `CuentaController.foto`, la misma cabecera que ya tiene el comprobante: `.header("X-Content-Type-Options", "nosniff")`.

Open Session In View

No es una vulnerabilidad sino la causa de la saturación brusca (ver capítulo 3 (Rendimiento)), pero conviene hacerlo en esta etapa:

```
spring.jpa.open-in-view=false
```

Riesgo: si algún controlador recorre una relación perezosa después de salir del servicio, aparece `LazyInitializationException`. Como `AGENTS.md` pide devolver DTOs armados en el servicio, debería haber pocos casos. **Correr todos los tests** y probar cada pantalla.

8.4 Etapa 3: cambios grandes

Dependencias: actualizar Spring Boot

Resuelve la mayoría de las 13 críticas y 39 altas del backend de una vez (Tomcat, Spring Framework, Spring Security, Jackson).

1. **Rama propia.** Pasar primero a la última versión de la línea 3.x y después evaluar la 4.x, que tiene más cambios.
2. **Spring Cloud tiene que acompañar.** La versión de `spring-cloud-dependencies` (hoy 2022.0.5, que es la de Boot 3.0 y 3.1) tiene que ser la que corresponde a la nueva versión de Spring Boot, según la tabla de compatibilidad de Spring Cloud.
3. **Revisar avisos de compilación y tests:** `./mvnw -B clean test`.
4. **Repetir Trivy** sobre la imagen nueva y **repetir estas pruebas de carga**: cambian Hibernate y Tomcat, y con ellos el rendimiento.

La versión exacta conviene elegirla al momento de hacerlo, con la que tenga soporte vigente en spring.io.

Frontend: reconstruir la imagen con una versión actual de `nginx:<versión>-alpine` y correr `npm audit fix`, que resuelve `follow-redirects`.

Archivos por partes

El arreglo de fondo del problema de memoria: que `ArchivoService` nunca tenga el archivo entero en un solo arreglo de bytes.

```
// Subir: del pedido al disco, cifrando de a pedazos
try (InputStream entrada = archivo.getInputStream();
     OutputStream salida = cryptoService.cifrarHacia(Files.newOutputStream(destino))) {
    entrada.transferTo(salida);
}
```

`cifrarHacia` escribiría primero el IV de 12 bytes y devolvería un `CipherOutputStream` con AES-GCM, **manteniendo el mismo formato que hoy** (`[IV][texto cifrado + tag]`), así los archivos que ya existen se siguen leyendo.

Para descargar hay un detalle: Java, al descifrar AES-GCM, retiene todo el contenido hasta verificar la etiqueta final. Igual se gana mucho, porque desaparece el buffer de memoria directa que causaba los `500`; para archivos de hasta 5 MB alcanza. Si en el futuro se permiten archivos grandes, conviene cifrar por bloques independientes.

Verificar: repetir `./limites.sh` con archivos y mirar con JFR que no aparezcan `OutOfMemoryError`.

C4 · Título del aporte cifrado

1. Una migración nueva cambia `aporte.titulo` y `aporte_version.titulo` a un tipo binario (como `contenido`).
2. `@Convert(converter = CifradoConverter.class)` en los dos campos.
3. **Migrar los títulos que ya existen**: Flyway con SQL no puede cifrar con la clave de OpenBao, así que hace falta una migración en Java o una tarea única que lea, cifre y guarde.

No hay ninguna consulta que filtre o busque por título, así que no se rompe ninguna búsqueda. **Para decidir con el equipo:** es un cambio de esquema con migración de datos.

C5 · Cerrar sesiones al cambiar la contraseña

1. Migración: columna `credenciales_cambiadas_en` en `usuario`, que se actualiza al restablecer la contraseña.
2. En `JwtAuthenticationFilter`, rechazar el token si su fecha de emisión (`iat`) es anterior a esa columna. El usuario ya se busca en cada pedido, así que no agrega consultas.

De paso, el filtro puede parsear el token **una sola vez** (hoy lo hace tres) y leer `subject` y `expiration` del mismo resultado.

C9 · Mails en segundo plano

`@EnableAsync` y `@Async` en los métodos de `EmailService`, con un grupo de hilos chico propio. Hay que cuidar que el mail se mande **después del commit** (ya existe ese patrón en `AvisoPorEmailService`), y registrar los envíos fallidos sin el contenido del mail.

D4 · Contenedores sin root

En `backend/Dockerfile`, en la etapa final:

```
RUN useradd --system --uid 10001 trama && chown -R trama /app
USER trama
```

Para el frontend, la variante `nginx-unprivileged` escucha en el 8080 sin root; hay que ajustar el puerto en el compose.

C7 · Código de recuperación

Con el límite por IP (C2) el riesgo baja mucho. Si además se quiere reforzar: códigos de 8 dígitos y un máximo de intentos **por cuenta y por día**, no solo por código.

8.5 Cómo saber que quedó arreglado

Hallazgo	Prueba que lo encontró	Resultado esperado después
D1	`nc` desde otra máquina	Puerto cerrado
D2	Login con la cuenta fija	`401`
C2	`limites.sh … logins`	`429` desde una IP y el backend sigue respondiendo
Archivos	`limites.sh … edicion textos archivos`	Subidas `201` después del pico

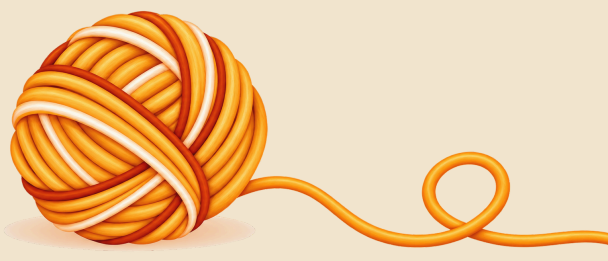
Hallazgo	Prueba que lo encontró	Resultado esperado después
Deadlocks	`limites.sh … edicion`	`409` en lugar de `500`, versiones intactas
C1	20 + 20 logins midiendo el tiempo	Medianas parecidas
C3	Error forzado en un test	Aparece en el log, sin datos del niño
Dependencias	Trivy	Sin críticas aplicables
Rendimiento	`correr.sh` en 1 CPU · 2 GB	Techo igual o mejor que 162 pedidos/s

TRAMA

ANEXOS

Anexos

Glosario, consultas SQL, módulos, excepciones y versiones



A. Glosario

Término	Qué significa en este informe
p95	El 95 % de los pedidos tardó menos que ese valor. Muestra la experiencia de las personas con peor suerte, que el promedio esconde
Mediana	La mitad de los pedidos tardó menos que ese valor
Pedidos por segundo	Cuántos pedidos HTTP atendió el backend en un segundo. Es la medida más estable de la capacidad de un servidor
Techo	El máximo de pedidos por segundo que alcanzó un servidor. Pasado ese punto, más gente se traduce en más espera
Usuario virtual	Una persona simulada por k6. Hace una acción cada 1 a 3 segundos, mucho más seguido que una persona real
VPS	Servidor virtual que se alquila por mes, con una cantidad fija de CPU, memoria y disco
BCrypt	Algoritmo con el que se guardan y verifican las contraseñas. Es lento a propósito (~100 ms por verificación) para que sea caro adivinarlas
JWT	El "pase" firmado que recibe la persona al iniciar sesión y que manda en cada pedido. En TRAMA dura 8 horas
Hikari	El administrador de conexiones a la base del backend. Por defecto mantiene 10 conexiones abiertas y los pedidos se turnan para usarlas
Open Session In View	Opción de Spring, activada por defecto, que mantiene la sesión con la base (y su conexión) abierta durante todo el pedido
N+1	Patrón en el que, para mostrar una lista de N elementos, se hace 1 consulta para la lista y N consultas más, una por elemento
Deadlock	Dos operaciones que se esperan mutuamente en la base. MySQL cancela una de las dos
Heap	La memoria donde Java guarda los datos del programa. La limpia el recolector de basura
Memoria directa	Memoria que Java usa por fuera del heap para leer y escribir archivos y red
Recolector de basura (GC)	Parte de Java que libera la memoria que ya no se usa. Algunos tipos (Serial) frenan toda la aplicación mientras trabajan
Binlog	Registro de cambios de MySQL, pensado para réplicas y recuperación. Por defecto se guarda 30 días
Java Flight Recorder (JFR)	Grabador incluido en Java que registra qué código usa la CPU, cuánta memoria pide y qué esperas hay
CVE	Identificador público de una vulnerabilidad conocida en un componente de software
Niño-año	Un niño con un año de datos cargados. Sirve para medir capacidad: 1.000 niño-años son 500 niños durante 2 años o 1.000 niños durante 1 año

B. Consultas SQL durante la saturación

1 CPU · 2 GB con 200 usuarios durante 110 segundos: **202.345 consultas para 7.353 pedidos HTTP** (unas 27 por pedido). Todas rápidas; el problema es la cantidad.

Consulta	Veces	Tiempo total	Promedio	Filas examinadas	Origen
Adjuntos de un aporte (`aporte_archivo` + `archivo`)	58.658	4,81 s	0,08 ms	0	N+1: una por aporte listado
`COMMIT`	15.588	2,32 s	0,15 ms	—	Cierre de cada transacción
Membresía del usuario en el proyecto	33.261	2,29 s	0,07 ms	33.261	Permisos, ~4,5 veces por pedido
`SET autocommit`	31.176	0,97 s	0,03 ms	—	Apertura de transacciones
Usuario por id	10.687	0,87 s	0,08 ms	10.687	Filtro JWT, en cada pedido
Aportes del proyecto	3.785	0,75 s	0,20 ms	102.170	Lista de aportes y bitácora
`SET SESSION TRANSACTION READ ONLY`	12.260	0,45 s	0,04 ms	—	Transacciones de lectura
Membresía (otra variante)	4.654	0,38 s	0,08 ms	4.654	Permisos
Confirmaciones de aportes	3.118	0,34 s	0,11 ms	0	Lista de aportes
`SET SESSION TRANSACTION READ WRITE`	12.260	0,31 s	0,03 ms	—	Transacciones de escritura

Estado de MySQL en la misma prueba: 99,9 % de lecturas desde memoria (1.049.720 pedidos al buffer, 1.323 lecturas de disco), 0 esperas por bloqueo de filas, 0 consultas lentas, 0 tablas temporales en disco, como máximo 11 conexiones abiertas.

C. Memoria y CPU por módulo de TRAMA

Porcentaje de las muestras de CPU de Java Flight Recorder, agrupadas por el módulo de TRAMA que estaba trabajando (1 CPU · 2 GB, cada flujo por separado):

Flujo	Módulo principal	Segundo	Tercero	Cuarto
Iniciar sesión	Login y cuentas 99 %	Filtro JWT 1 %	—	—
Inicio	Filtro JWT 24 %	Tomcat 23 %	Proyectos 21 %	Fuera de TRAMA 17 %
Ver aportes	Aportes 40 %	Permisos 13 %	Tomcat 12 %	Filtro JWT 12 %
Bitácora	Aportes 40 %	Permisos 12 %	Filtro JWT 11 %	Tomcat 11 %

Flujo	Módulo principal	Segundo	Tercero	Cuarto
Cargar aporte	Filtro JWT 24 %	Tomcat 21 %	Aportes 16 %	Fuera de TRAMA 15 %
Editar aporte	Tomcat 28 %	Aportes 24 %	Filtro JWT 17 %	Permisos 9 %
Subir archivo	Tomcat 69 %	Filtro JWT 8 %	Permisos 8 %	Archivos 6 %

Y por capa técnica:

Flujo	Capa principal	Segunda	Tercera	Cuarta
Iniciar sesión	BCrypt 98 %	Hibernate 1 %	—	—
Inicio	Spring 23 %	Hibernate 16 %	Tomcat 15 %	JWT (jjwt) 12 %
Ver aportes	Driver de MySQL 30 %	Hibernate 21 %	Spring 16 %	Otras 10 %
Bitácora	Driver de MySQL 23 %	Hibernate 23 %	Spring 15 %	Jackson 11 %
Cargar aporte	Spring 23 %	Hibernate 22 %	Driver de MySQL 13 %	Tomcat 13 %
Editar aporte	Hibernate 29 %	Spring 18 %	Tomcat 13 %	Otras 9 %
Subir archivo	Tomcat 65 %	Hibernate 11 %	Spring 10 %	Driver de MySQL 5 %

D. Excepciones registradas en la falla de subidas

Grabación de Java Flight Recorder con el registro de excepciones activado, durante la reproducción del caso de archivos (2 CPU · 4 GB):

Excepción	Veces	Qué es
`java.lang.OutOfMemoryError` (memoria directa)	806	La causa de los `500` en subidas (622, `ArchivoService` línea 142) y descargas (184, línea 211)
`jakarta.servlet.ServletException`	806	El envoltorio con el que el error llega a `GlobalExceptionHandler`
`MySQLTransactionRollbackException`, `LockAcquisitionException`, `OptimisticLockException`	206 cada una	Los *deadlocks* de la edición simultánea
`InvocationTargetException`, `FileAlreadyExistsException`, `NumberFormatException`, `ClassNotFoundException`	miles	Internas de Java, Spring e Hibernate; normales y atrapadas por las propias librerías

E. Entorno y versiones

Pieza	Versión
Backend	Spring Boot 3.1.11 · Spring Framework 6.0.19 · Spring Security 6.1.9 · Tomcat 10.1.20 · Hibernate 6 · Java 21.0.13
Base	MySQL 8.0.46
Generador de carga	k6 0.54.0
Escáner	Trivy 0.57.1
Este PDF	Generado con Chromium 146 a partir de `docs/pruebas-de-carga/`